# DIGT2107: Practice of Software Development

**Project Deliverable 3.1:**

Design and GUI- Submission

**Project Name:**

SmartBudget – Personal Finance Tracker

**Team Number:**

Team #3

**Team Members:**

Mohsen Pasdar, Muhammed Ahmed, Alia Hagi-Dhaffe

**Date:**

February 15, 2026

# Table of Contents


1 Introduction ..... 1

2 System Design Overview (SDD) ..... 1

2.1 Architectural Style and Layering ..... 1

2.2 Layer Responsibilities ..... 2

2.3 Components and Responsibilities ..... 3

2.4 Validation and Error Handling Strategy ..... 3

2.5 Design Coverage of Key Use Cases ..... 3

3 UML Class Diagrams ..... 3

3.1 High-Level Class Diagram (System-Wide View) ..... 4

3.2 Class Diagram A — Transactions Module ..... 4

3.3 Class Diagram B — Budgets and Alerts Module ..... 5

3.4 Class Diagram C — Reports Module ..... 6

3.5 Cross-Cutting Classes Used Across Modules ..... 7

4 Sequence Diagrams ..... 8

4.1 SD-1 — UC-1 Record Transaction (Add Expense/Income) ..... 8

4.2 SD-2 — UC-4 Set Monthly Budget + UC-6 Overspending Alert Check ..... 10

4.3 SD-3 — UC-5 View Summary Report ..... 11

5 Activity Diagrams ..... 12

5.1 UC-1 Record Transaction (Add) ..... 12

5.2 UC-2 Amend Transaction ..... 13

5.3 UC-3 View & Filter Transactions ..... 15

5.4 UC-4 Set Monthly Budget ..... 15

5.5 UC-4 Edit or Delete Monthly Budget ..... 17

5.6 UC-5 View Summary Report ..... 18

5.7 Load Saved Data on Startup ..... 19

6 Updated Backlog with Design Stories ..... 19

6.1 Backlog Format and Prioritization Approach ..... 20
6.2 Updated Backlog (Design Stories) ..... 20
6.3 Notes on Current Progress (Iteration 3.1) ..... 21
7 GUI Implementation (JavaFX) + Screenshots ..... 22
7.1 UI Structure and Navigation ..... 22
7.2 Transactions UI ..... 22
7.2.1 Transactions Page ..... 22
7.2.2 Add Transaction Modal ..... 23
7.2.3 Edit Transaction Modal ..... 24
7.2.4 Delete Transaction Confirmation ..... 25
7.3 Budgets UI ..... 26
7.3.1 Budgets Page ..... 26
7.3.2 Add Budget Modal ..... 27
7.3.3 Edit Budget Modal ..... 28
7.3.4 Delete Budget Confirmation ..... 29
7.3.5 Reports UI ..... 30
7.3.6 Summary Report Page ..... 30
7.4 Notes on Current GUI Scope (Iteration 3.1) ..... 32
8 Preliminary Testing + JaCoCo Coverage ..... 32
8.1 Testing Approach ..... 33
8.2 Manual GUI Smoke Test Results ..... 33
8.3 Automated Unit Testing Summary ..... 33
8.4 JaCoCo Coverage Summary ..... 34
8.5 Issues Identified and How They Will Be Addressed ..... 34
8.5.1 Main coverage gaps identified: ..... 34
8.5.2 Planned actions for the next development cycle: ..... 35
9 Code Repository ..... 35

9.1 Repository Link: ..... 35

10 Conclusion ..... 35

# 1 Introduction

SmartBudget is a personal finance application that helps users record daily transactions, set monthly budgets by category, and view summary reports to better understand their spending habits. The goal of the system is to make budgeting simple and practical by giving users a clear view of where their money goes, and by supporting better decision-making through budget tracking and spending summaries.

This deliverable focuses on the software design and the initial implementation of SmartBudget. In this phase, our team completed the main design documents (including class diagrams, sequence diagrams, and activity diagrams) and implemented the core parts of the application based on these designs. We developed the project code base with working domain and service classes, connected the main functionality to the GUI, and wrote unit tests to verify key business logic. Overall, the system is functional and supports the main user workflows, with only minor improvements and refinements still needed.

**Important limitation**: data persistence is not implemented yet, so the application data is not saved after the program closes. This will be addressed in the next deliverable. Despite this limitation, the current version of SmartBudget runs correctly during a session and demonstrates the intended features and interactions of the system.

# 2 System Design Overview (SDD)

## 2.1 Architectural Style and Layering

SmartBudget follows a layered architecture to keep the user interface, business logic, domain entities, and persistence concerns separated. This design improves maintainability and makes the system easier to test and extend. The system is organized into four main layers: **UI (View + Controller), Service (Business Logic), Domain Model (Entities/Results),** and **Persistence (Storage).**

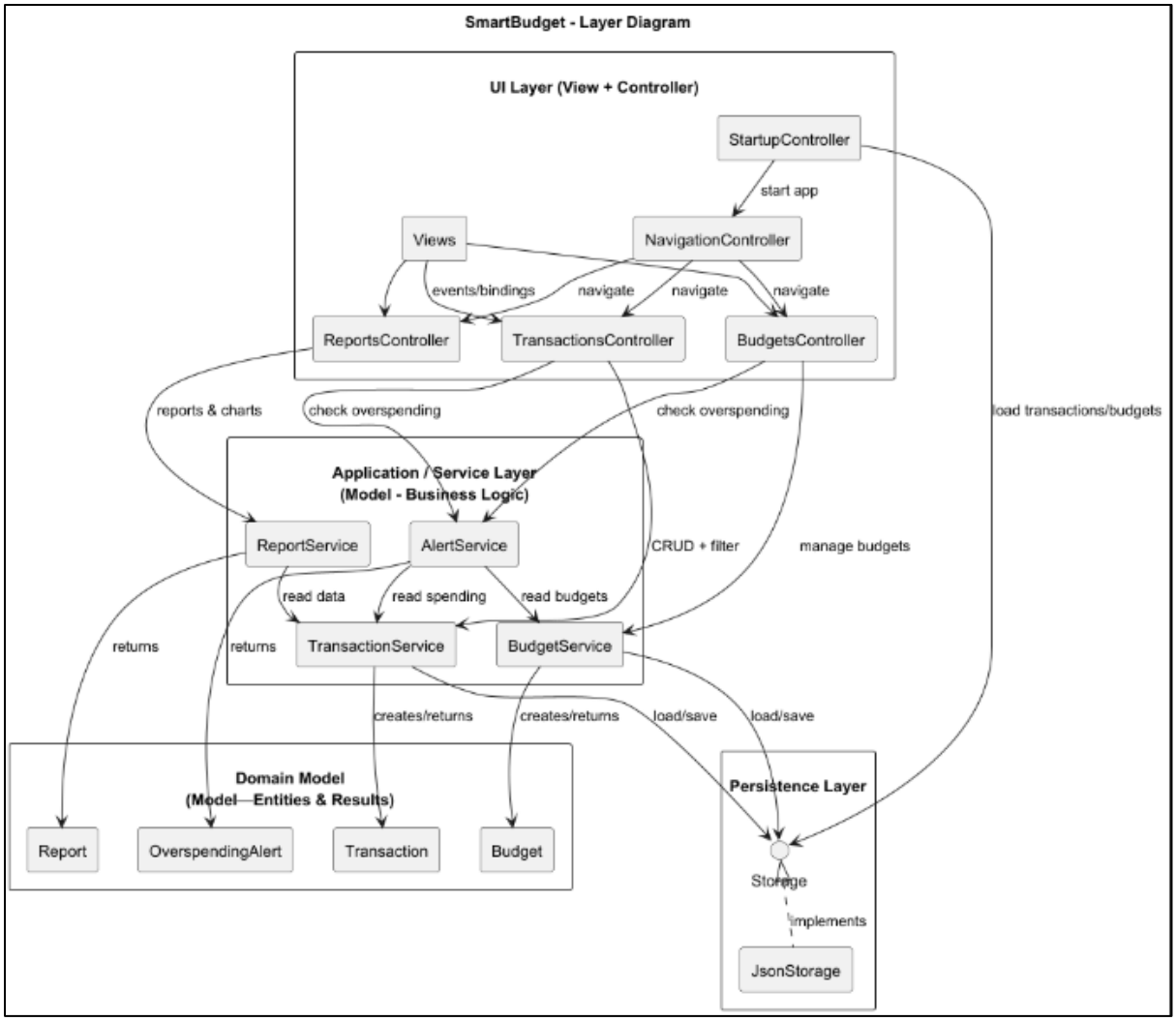


*Figure 2-1: Layer diagram*

## 2.2 Layer Responsibilities

- **UI Layer (Views + Controllers):** Handles user interactions and navigation. Controllers collect inputs, call services, and update the screen.
- **Service Layer:** Contains the core business rules such as adding transactions, managing budgets, generating reports, and checking overspending alerts.
- **Domain Model:** Represents the system's main data objects (e.g., Transaction, Budget) and report-related result objects (e.g., SummaryReport).
- **Persistence Layer:** Abstracts saving/loading through a **Storage** interface. A JSON-based storage class is designed (JsonStorage), but persistence is not finalized in this iteration.

## 2.3 Components and Responsibilities

The application is centered around three main functional modules:

- **Transactions module:** Records, edits, and filters transactions through TransactionsController and TransactionService.
- **Budgets + Alerts module:** Sets monthly budgets through BudgetsController and BudgetService, and checks overspending through AlertService.
- **Reports module:** Generates summary reports through ReportsController and ReportService.

## 2.4 Validation and Error Handling Strategy

Input validation is handled consistently using a Validator component. Controllers pass user inputs to services, and services call the validator before creating or updating domain objects. If validation fails, the service returns an error response (or throws a validation exception) and the controller displays a clear message to the user.

## 2.5 Design Coverage of Key Use Cases

The design documentation covers the main workflows required by the system, including recording and editing transactions, viewing/filtering transactions, setting/editing budgets, generating summary reports, and loading saved data on startup. The sequence and activity diagrams show both the main paths and important decision points such as invalid input or user cancellation.

# 3 UML Class Diagrams

This section presents the UML class diagrams for SmartBudget. The diagrams describe the system's static structure, including the main classes, their attributes and operations, and how they relate to each other through associations and dependencies. We include a high-level class diagram to give an overall picture of the architecture, followed by three detailed diagrams that break the design into the main modules: Transactions, Budgets & Alerts, and Reports. Together, these diagrams show how responsibilities are separated across controllers, services, domain entities, and storage-related components.

## 3.1 High-Level Class Diagram (System-Wide View)

The high-level class diagram presents the overall structure of SmartBudget, showing how the main controllers interact with their services, how services use domain entities, and how persistence is abstracted through a storage component. This view highlights the separation between UI logic and business logic.

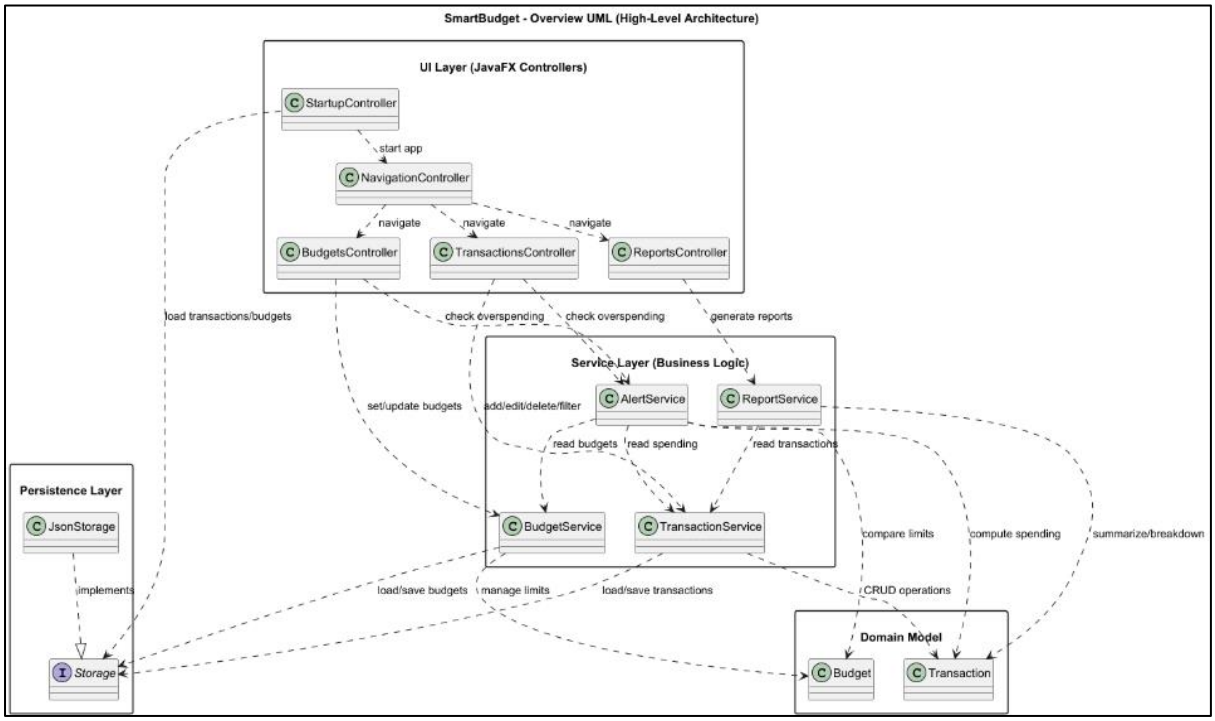


*Figure 3-1: High-Level UML Class Diagram*

## 3.2 Class Diagram A — Transactions Module

This diagram focuses on transaction-related functionality, including recording and managing transactions. Key classes include *TransactionsController*, *TransactionService*, and the *Transaction* domain entity. It also shows supporting classes such as *TransactionFilter* for filtering logic and shared components like *Validator*, *AlertService*, and *JsonStorage*.

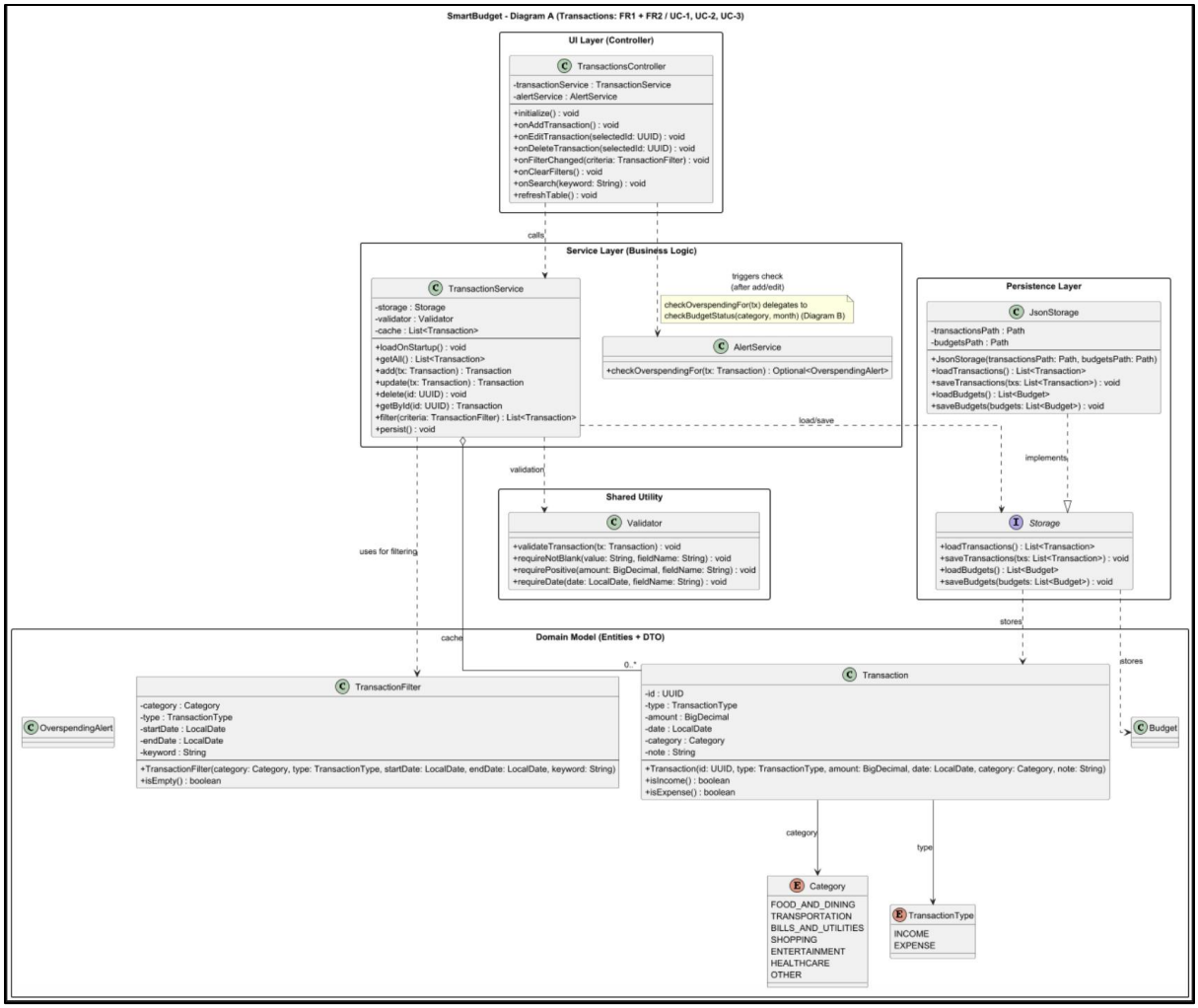


*Figure 3-2: UML Class Diagram A — Transactions*

## 3.3 Class Diagram B — Budgets and Alerts Module

This diagram focuses on setting budgets and checking overspending alerts. The *BudgetsController* works with *BudgetService* to create or update monthly budgets, while *AlertService* compares budget limits with transaction totals. Shared dependencies include *TransactionService*, *Validator*, and storage (*JsonStorage* / *Storage* interface).

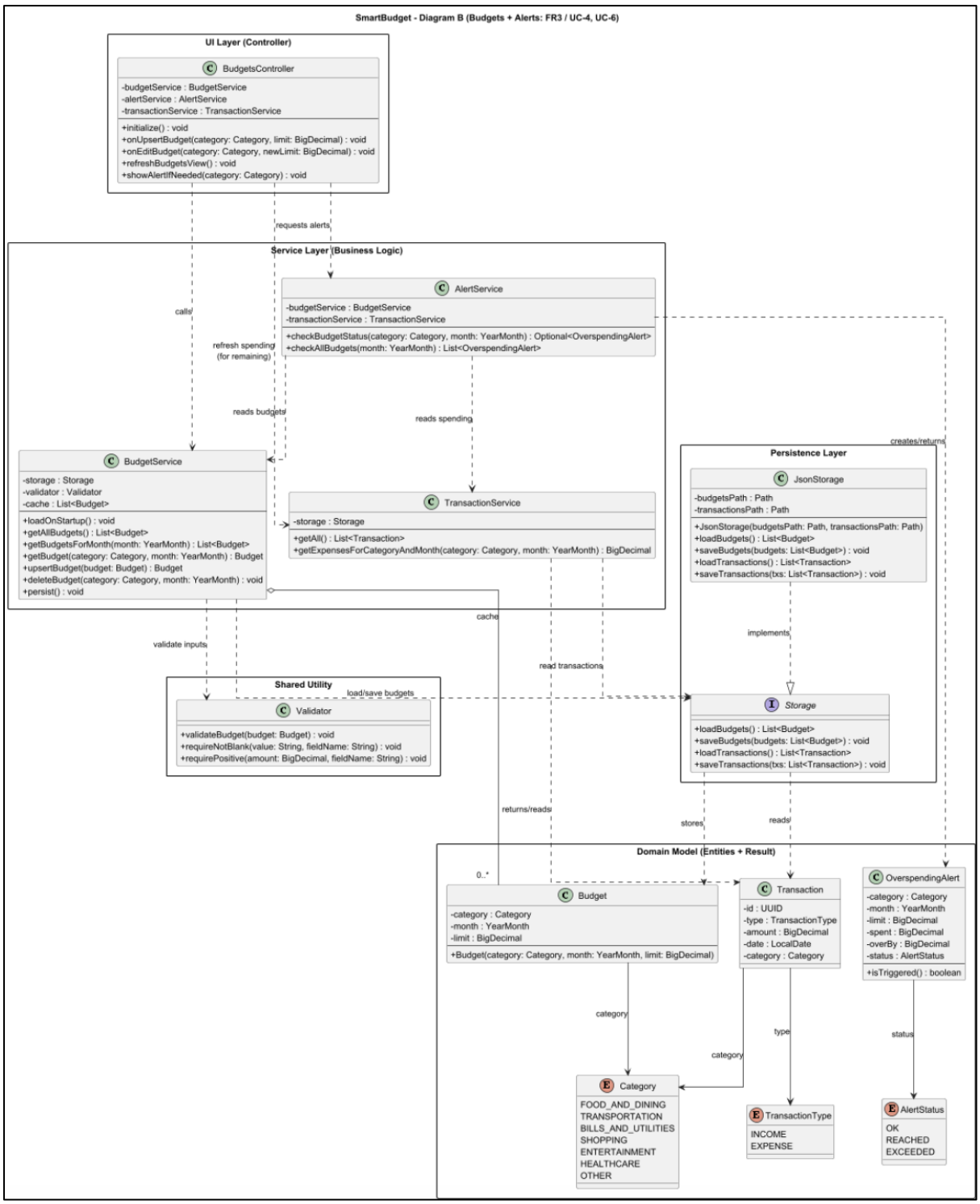


*Figure 3-3: UML Class Diagram B — Budgets and Alerts*

## 3.4 Class Diagram C — Reports Module

This diagram focuses on reporting features. *ReportsController* requests reports from *ReportService*, which uses *TransactionService* to retrieve transactions in a selected period.

Report results are represented through reporting objects such as *SummaryReport*, *CategoryTotal*, and *ReportResult*, which structure the report output in a clean and testable way.

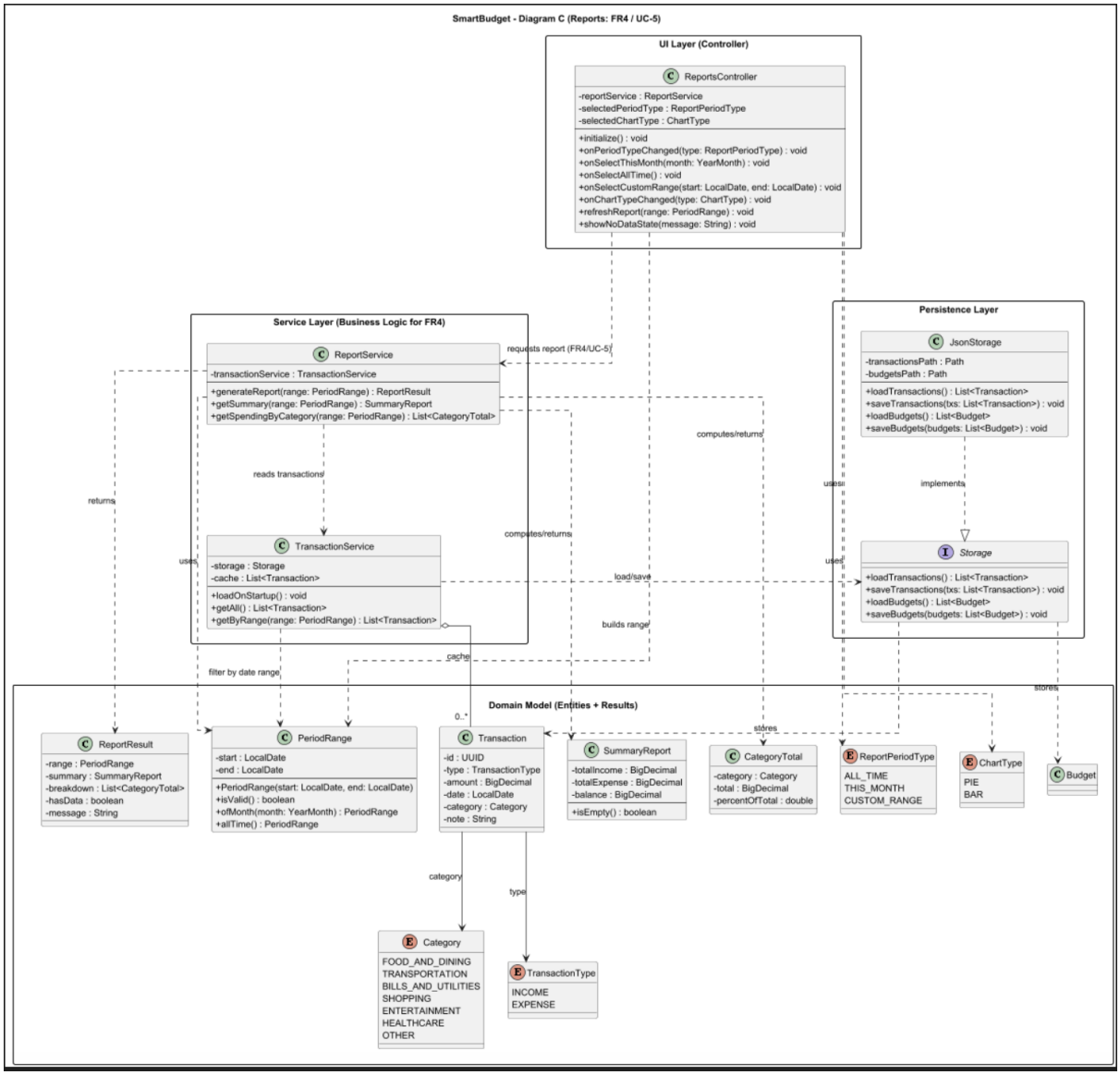


*Figure 3-4: UML Class Diagram C — Reports*

## 3.5 Cross-Cutting Classes Used Across Modules

Several components are shared across multiple modules:

- **Validator:** Central validation rules used before creating/updating objects.
- **JsonStorage / Storage:** Abstraction for saving and loading data.
- **AlertService:** Used by both transactions and budgets to detect overspending situations.

# 4 Sequence Diagrams

This section includes the sequence diagrams for SmartBudget, which illustrate the system's runtime behavior for key workflows. Each diagram shows the step-by-step interaction between the user interface (controllers), service layer, and domain objects, including how validation and results are handled. The selected sequence diagrams represent critical use cases and demonstrate how the system processes user requests from the moment the user triggers an action in the GUI to the point where the business logic completes and a response is returned to the UI.

## 4.1 SD-1 — UC-1 Record Transaction (Add Expense/Income)

This sequence diagram shows how a user records a new transaction. The controller collects form inputs, the service validates them, a new *Transaction* is created, and the updated data is returned. The diagram also includes an alternate path for invalid input, where the user receives an error message instead of saving the transaction.

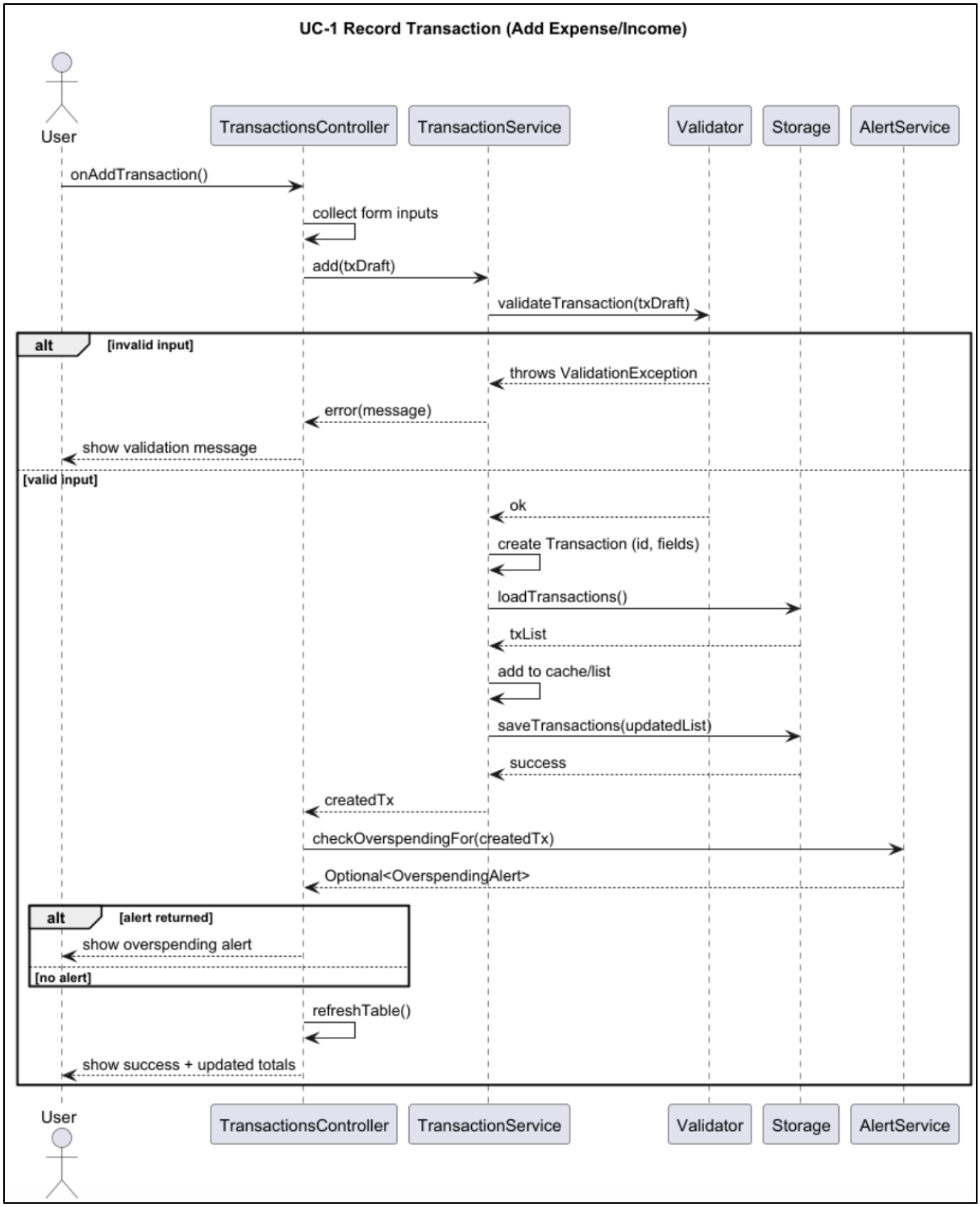


*Figure 4-1: Sequence Diagram — UC-1 Record Transaction*

## 4.2 SD-2 — UC-4 Set Monthly Budget + UC-6 Overspending Alert Check

This sequence diagram shows how a budget is created or updated for a given month and category. After validation, the budget is saved/updated through the service. The flow then triggers overspending checks by comparing spending (from transactions) against the budget limit and returning an alert result to the UI if needed.

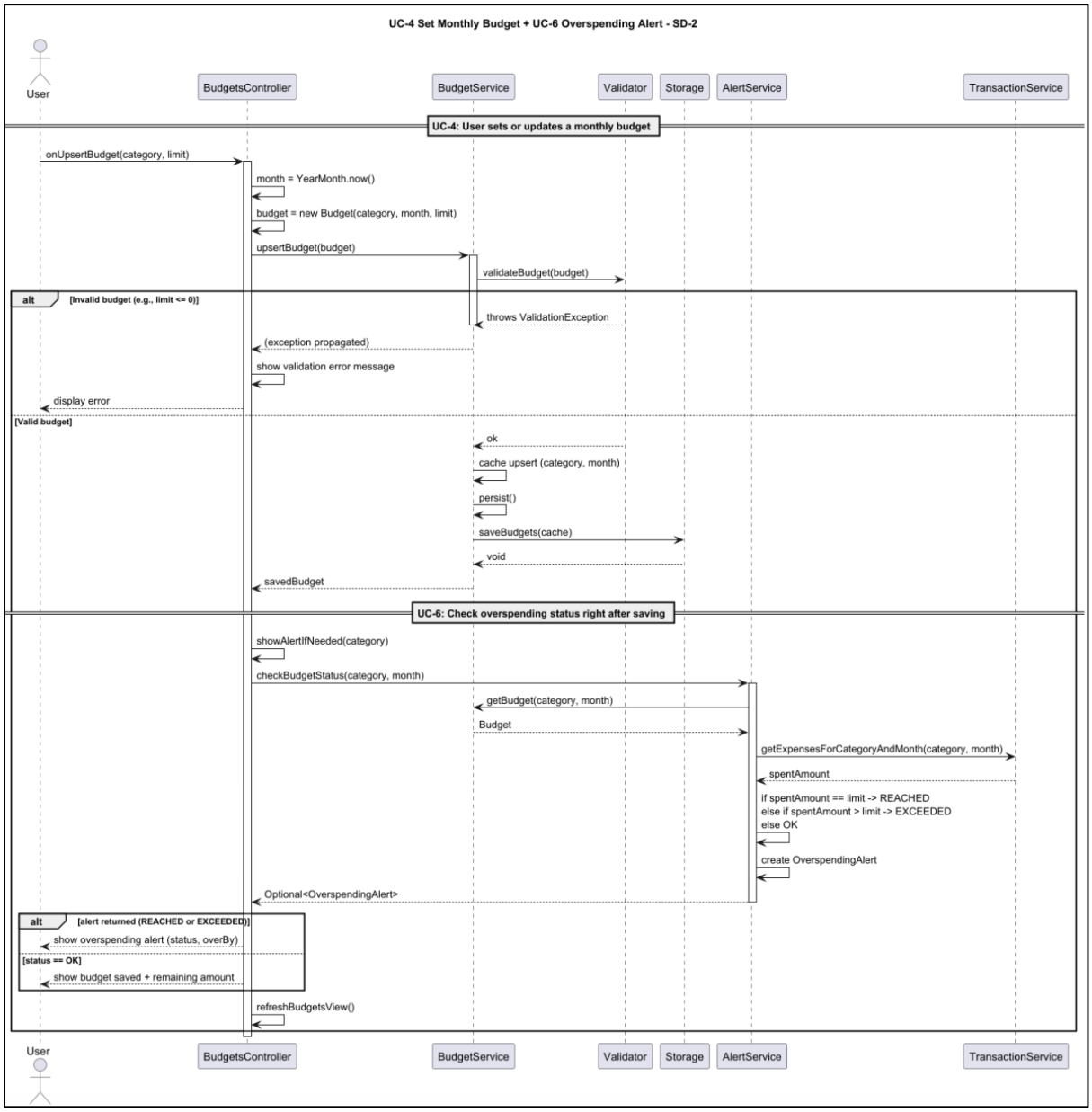


*Figure 4-2: Sequence Diagram — UC-4 Set Monthly Budget + UC-6 Alert Check*

## 4.3 SD-3 — UC-5 View Summary Report

This sequence diagram shows how a user generates a summary report for a selected time range. The controller creates a *PeriodRange*, calls *ReportService*, and the report service retrieves filtered transactions through *TransactionService*. The output is returned as a report result object that the UI can display.

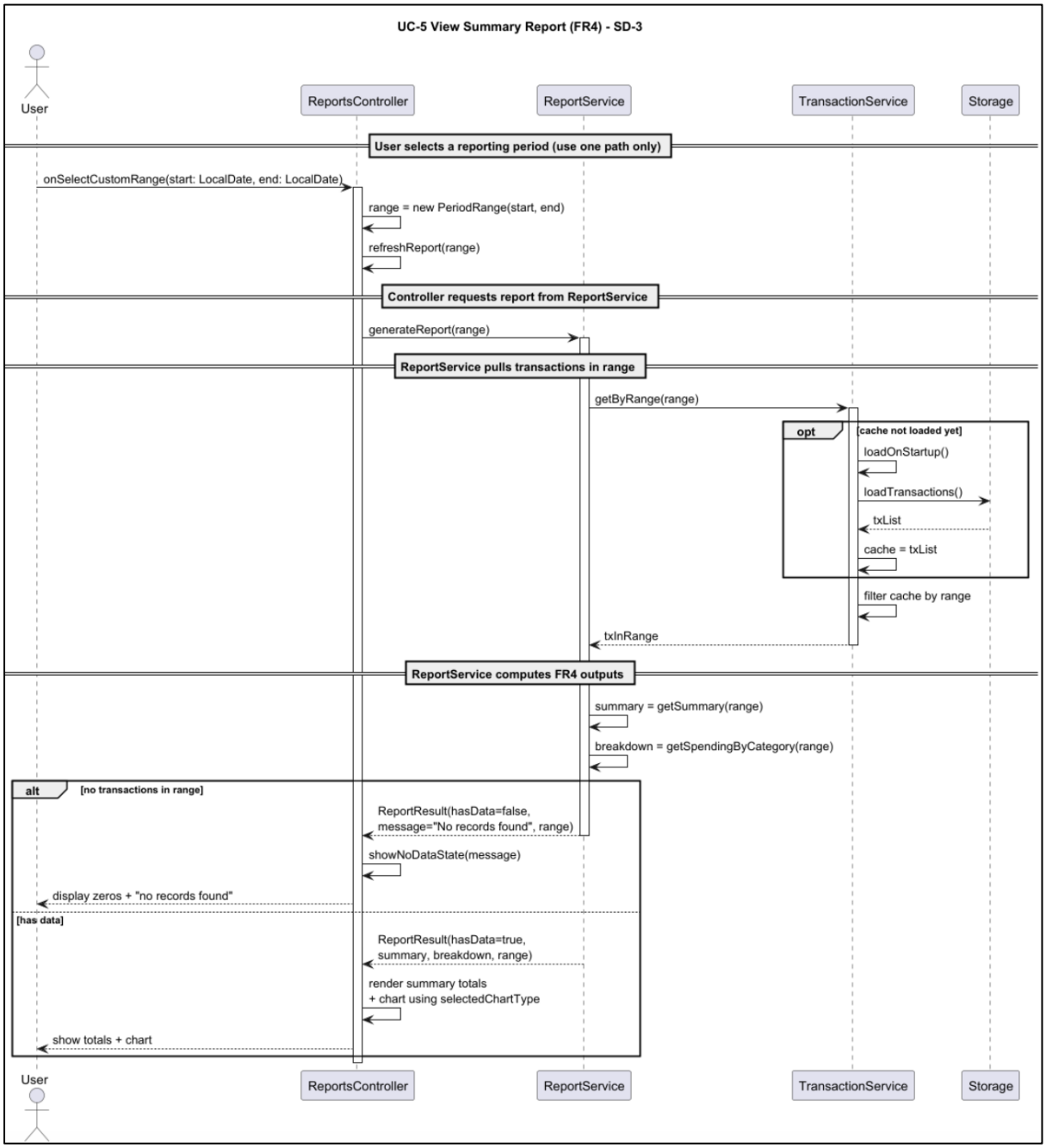


*Figure 4-3: Sequence Diagram — UC-5 View Summary Report*

# 5 Activity Diagrams

This section provides the activity diagrams for SmartBudget. These diagrams describe the control flow of major user workflows, including actions, decision points, and alternate paths such as invalid inputs or cancellation. The activity diagrams complement the sequence diagrams by focusing on the overall process logic rather than object-to-object messages. Together, they show how the application guides the user through tasks like recording or amending transactions, setting and updating budgets, generating reports, and initializing the system on startup.

## 5.1 UC-1 Record Transaction (Add)

This activity diagram shows how a user opens the add-transaction modal, enters transaction data, and submits it. It also includes decision points for canceling the modal and handling invalid input before saving.

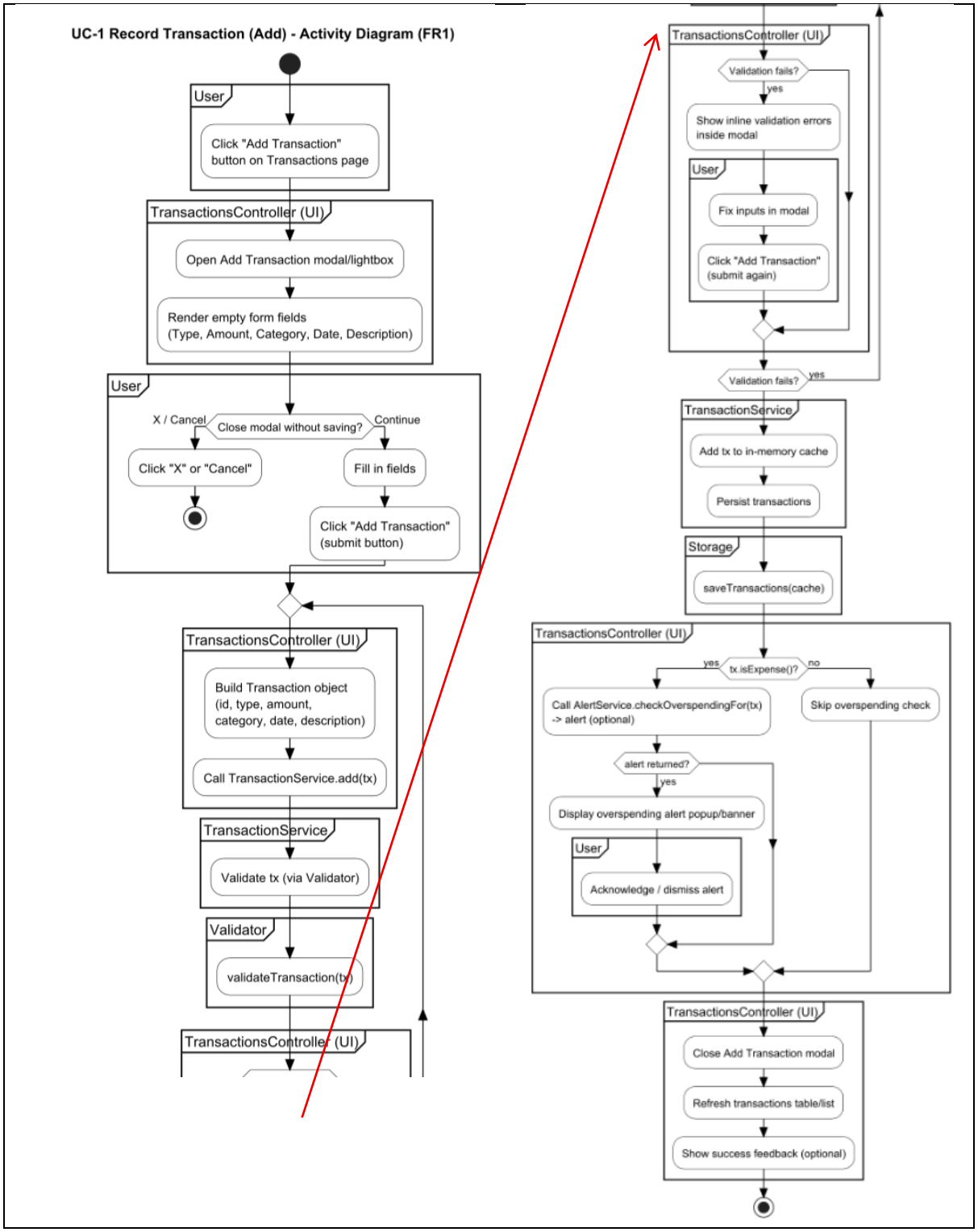


*Figure 5-1: Record Transaction (UC-1 FR1).puml screenshot Figure 6.1: Activity Diagram — UC-1 Record Transaction (Add)*

## 5.2 UC-2 Amend Transaction

This activity diagram shows editing an existing transaction, validating the updated fields, and saving changes. It also covers cancellation and validation failure paths.

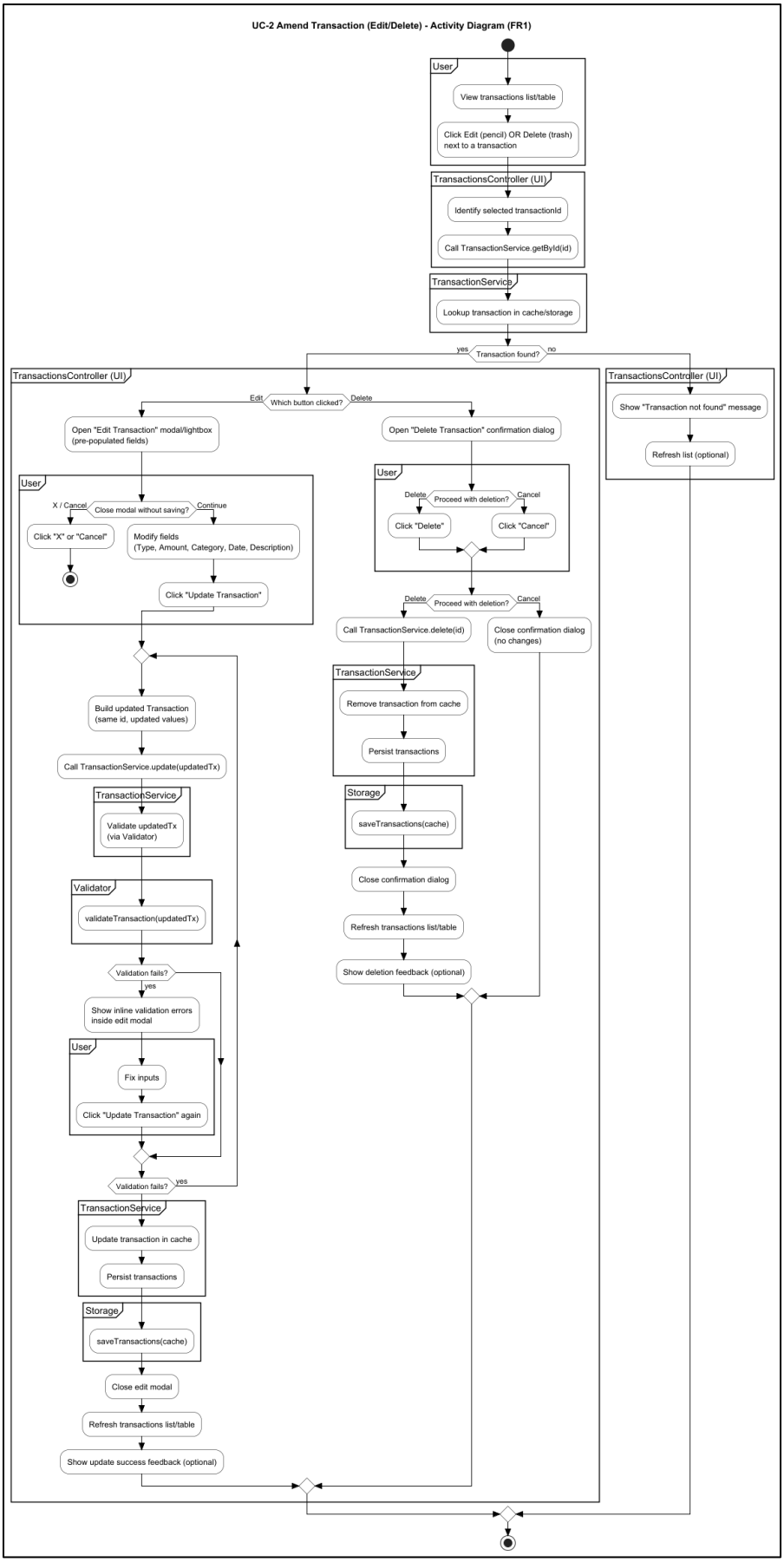


*Figure 5-2: Amend Transaction (UC-2 FR1*

## 5.3 UC-3 View & Filter Transactions

This activity diagram shows how the user applies filters (such as category/date/type) and how the system refreshes the displayed list based on filter inputs. It also reflects the workflow for clearing or adjusting filters.

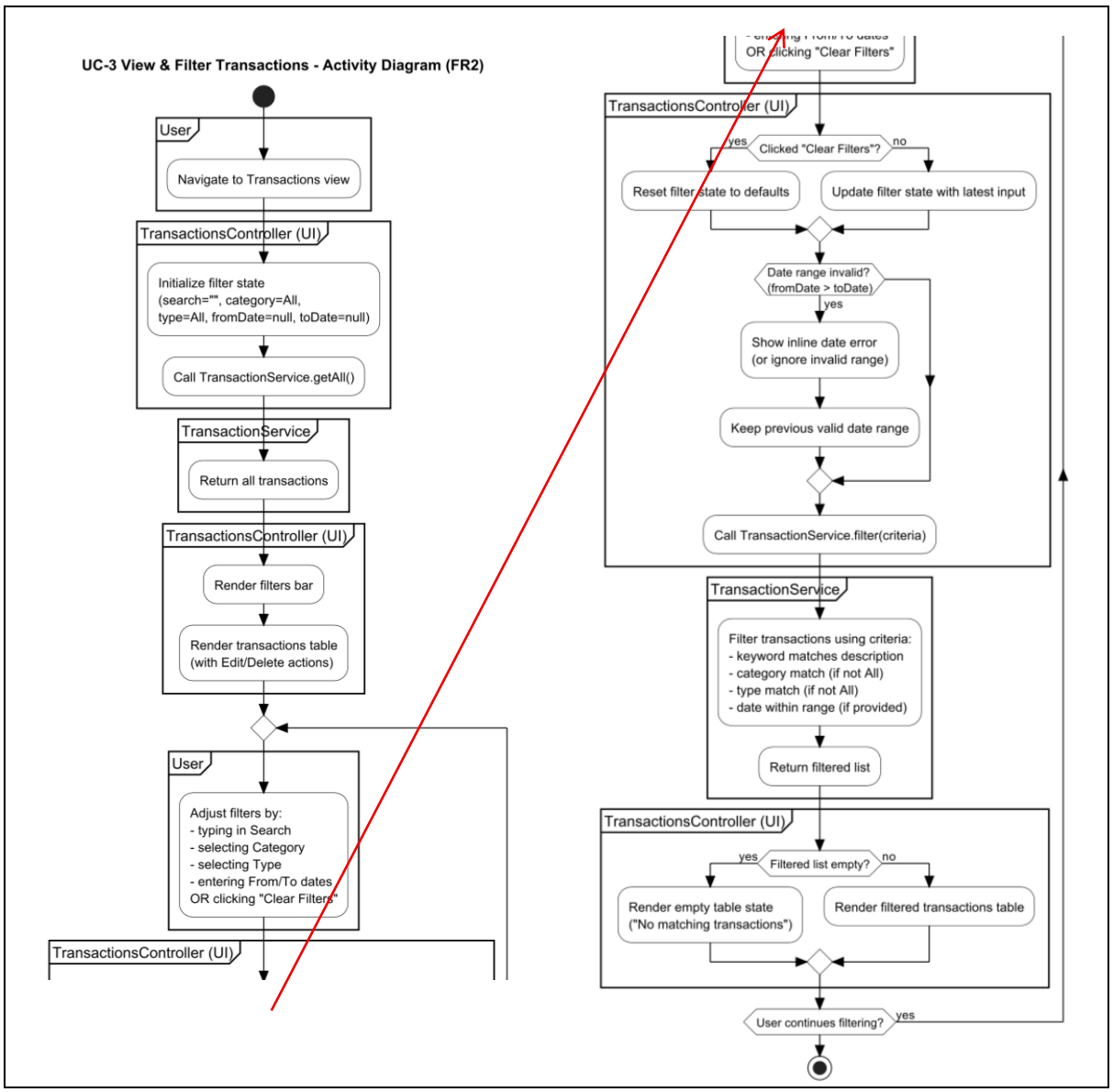


*Figure 5-3: Activity Diagram — UC-3 View & Filter Transactions*

## 5.4 UC-4 Set Monthly Budget

This activity diagram shows entering a monthly limit for a category, validating inputs, saving the budget, and triggering overspending checks when applicable.

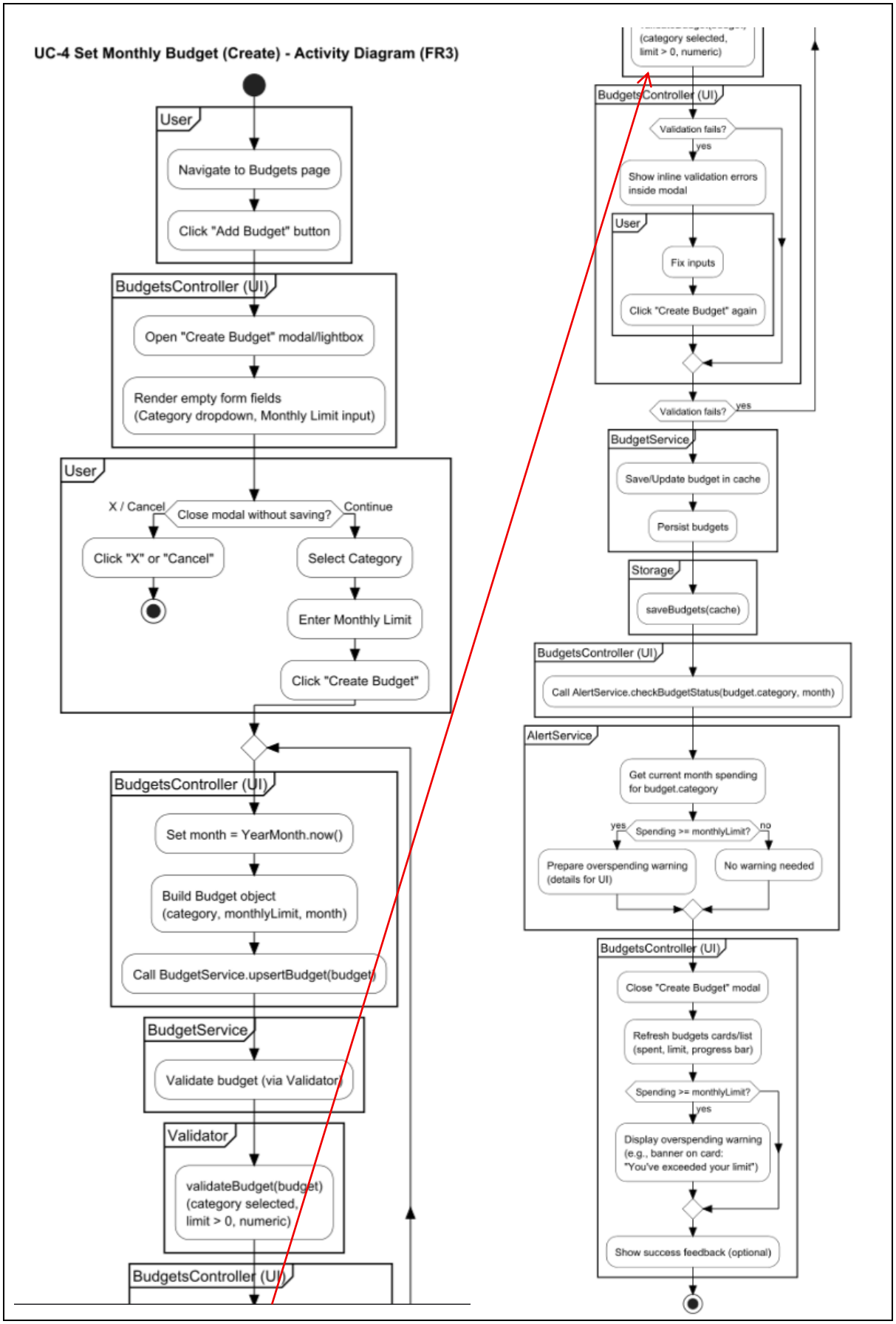


*Figure 5-4: Activity Diagram — UC-4 Set Monthly Budget*

## 5.5 UC-4 Edit or Delete Monthly Budget

This activity diagram shows the workflow for updating an existing budget or deleting it. It includes decision points for edit vs delete, confirmation steps, and validation errors.

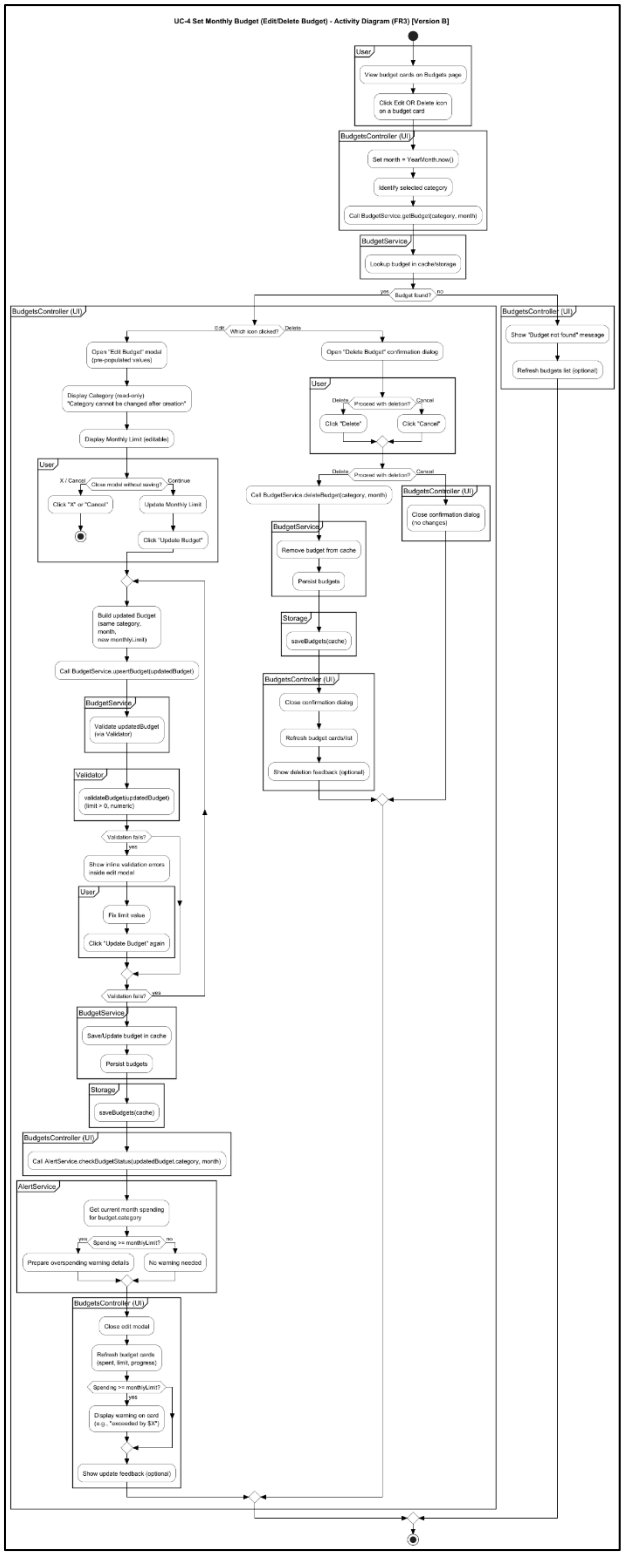


*Figure 5-5: UC-4 Edit/Delete Monthly Budget*

## 5.6 UC-5 View Summary Report

This activity diagram shows choosing a report period, generating the report, and displaying category totals and summary results. It covers alternate paths such as empty results or invalid ranges (if applicable in your logic).

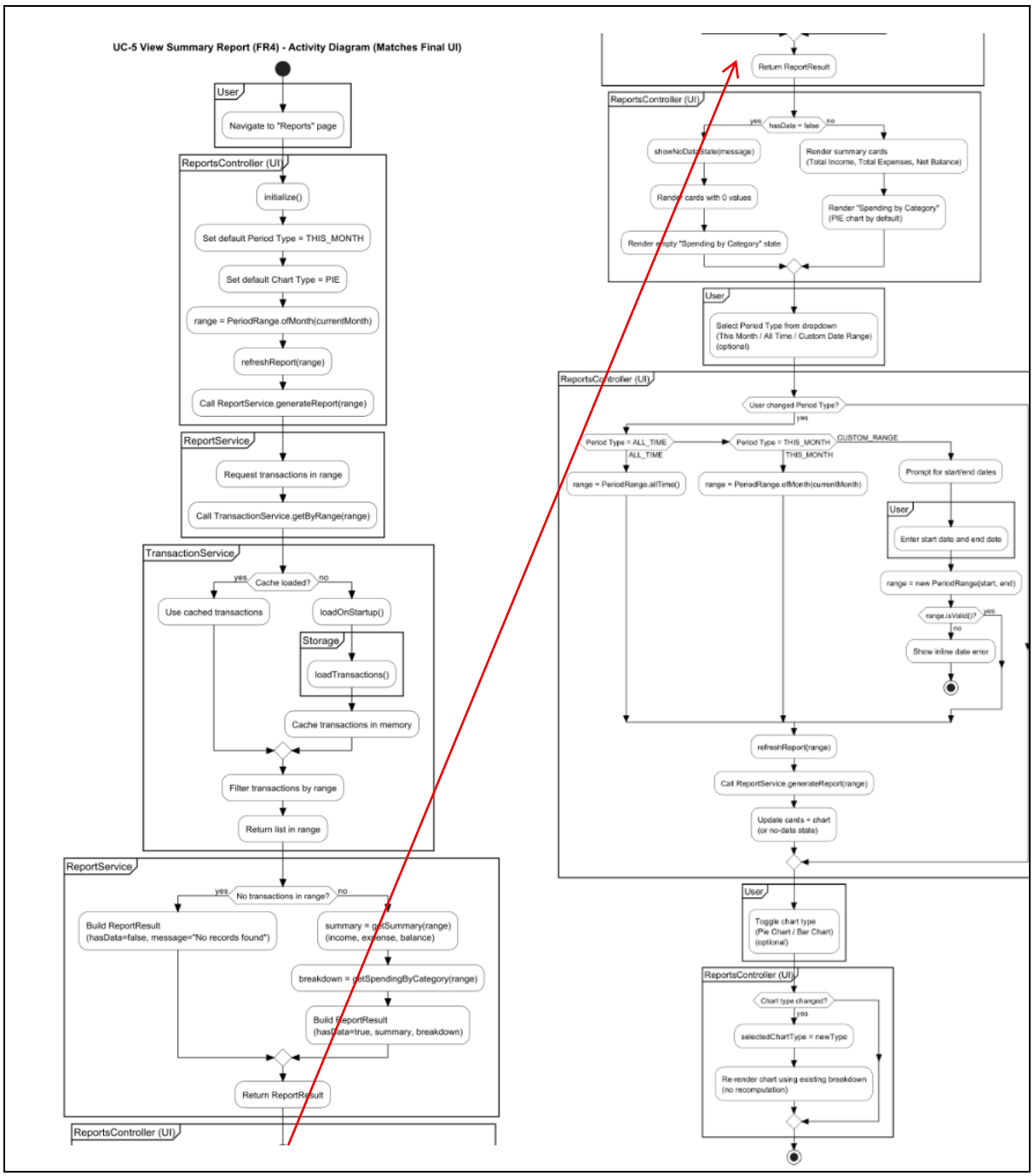


*Figure 5-6: Activity Diagram — UC-5 View Summary Report*

## 5.7 Load Saved Data on Startup

This activity diagram shows the startup flow of the application: initialization of controllers/services, calling the startup controller to load saved data, and landing on the Transactions page. This workflow supports the planned persistence behavior (even if persistence is completed in the next deliverable).

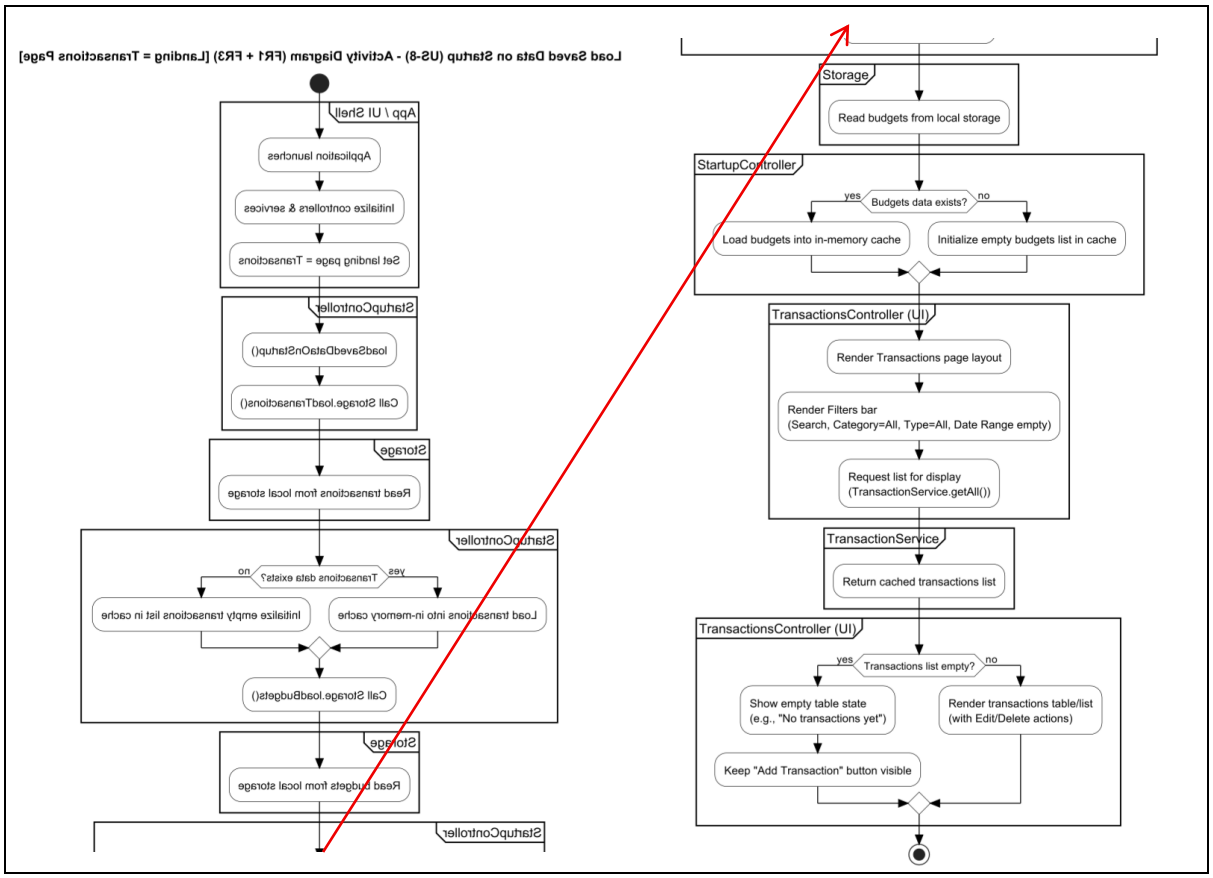


*Figure 5-7: Activity Diagram — Load Saved Data on Startup*

# 6 Updated Backlog with Design Stories

This section presents the updated backlog for SmartBudget for Iteration 3.1 and the upcoming development cycle. The backlog is based on the project's core user stories defined earlier (Deliverable 2.2), and it has been refined into more specific design and implementation stories that reflect the current system architecture (UI → Services → Domain → Persistence) and the work completed so far.

Because most core features are now functional, this backlog is split into:

- **Completed design/implementation stories** (Done in Iteration 3.1), and

- **Remaining / next-cycle stories** (To Do), focusing mainly on persistence, missing test coverage, and a few UI improvements.

## 6.1 Backlog Format and Prioritization Approach

Backlog items are prioritized based on:

- **Dependency order** (persistence must exist before persistence testing, for example),
- **Core system value** (features that affect correct app behavior are highest priority),
- **Quality improvements** (test coverage, validation, and UI polish).

## 6.2 Updated Backlog (Design Stories)

*Table 6-1: Updated Backlog (Design Stories)*

| ID | Design Story (Developer Story Format) | Related User Story | Priority | Dependency | Status | Target |
|---|---|---|---|---|---|---|
| **DS-01** | As a developer, I have a layered architecture design (UI, Services, Domain, Persistence) so that responsibilities are separated and the system is easier to maintain and test. | All | High | None | Done | Iteration 3.1 |
| **DS-02** | As a developer, I have implemented the Transactions module (add/edit/delete transaction workflows) so that users can manage their transactions correctly through the service and domain layers. | US-1, US-2 | High | DS-01 | Done | Iteration 3.1 |
| **DS-03** | As a developer, I have implemented transaction viewing and filtering so that users can search and review their transactions based on criteria such as category, type, and date range. | US-3 | High | DS-02 | Done | Iteration 3.1 |
| **DS-04** | As a developer, I have implemented the Budgets module (set/edit/delete monthly budgets) so that users can plan spending limits per category through the service and domain layers. | US-4 | High | DS-01 | Done | Iteration 3.1 |
| **DS-05** | As a developer, I have integrated overspending alert logic into budget / transaction workflows so that the system can detect when spending exceeds a budget and notify the UI when needed. | US-5 | Medium | DS-04, DS-02 | Done (partial) | Iteration 3.1 |
| **DS-06** | As a developer, I have implemented summary report generation for a selected period so that users can view totals and category breakdowns for a custom date range. | US-6 | High | DS-02 | Done | Iteration 3.1 |

| | | | | | | |
|---|---|---|---|---|---|---|
| **DS-07** | As a developer, I have implemented JavaFX page navigation (Transactions, Budgets, Reports) so that users can move between main features using a consistent UI structure. | All | High | DS-01 | Done | Iteration 3.1 |
| **DS-08** | As a developer, I have implemented JavaFX modals and confirmations for transactions and budgets so that create / update / delete actions are clear, validated, and safe from accidental changes. | US-1, US-2, US-4 | High | DS-07 | Done | Iteration 3.1 |
| **DS-09** | As a developer, I have written initial unit tests for domain and service logic so that core business rules are verified independently from the GUI. | All | High | DS-02, DS-04, DS-06 | Done (partial) | Iteration 3.1 |
| **DS-10** | As a developer, I have generated a JaCoCo coverage report so that we can measure test execution coverage and identify untested modules for the next cycle. | All | High | DS-09 | Done | Iteration 3.1 |
| **DS-11** | As a developer, I have implemented a persistence layer (Storage interface + JSON save/load + integration) so that application data is saved and restored across sessions. | US-8 | High | DS-01 | To Do | Iteration 3.2 |
| **DS-12** | As a developer, I have implemented loading saved data on startup so that users automatically see their saved transactions and budgets when launching the application. | US-8 | High | DS-11 | To Do | Iteration 3.2 |
| **DS-13** | As a developer, I have written unit tests for persistence (save/load correctness + restart regression tests) so that persistence behavior is reliable and does not break existing features. | US-8 | High | DS-11, DS-12 | To Do | Iteration 3.2 |
| **DS-14** | As a developer, I have added unit tests for AlertService so that overspending alert rules (including boundary cases) are verified and UC-6 is validated. | US-5 | Medium | DS-05 | To Do | Iteration 3.2 |
| **DS-15** | As a developer, I have added unit tests for ReportService and PeriodRange so that report calculations, date range validation, and empty-result behavior are correct and UC-5 is validated. | US-6 | Medium | DS-06 | To Do | Iteration 3.2 |
| **DS-17** | As a developer, I have improved the transaction form UI to filter categories by transaction type (Income vs Expense) so that users only see relevant categories and input errors are reduced. | US-1 | Medium | DS-08 | To Do | Iteration 3.2 |

## 6.3 Notes on Current Progress (Iteration 3.1)

- The backlog shows that SmartBudget has reached a strong implementation state: core transaction, budget, and report workflows are functional, supported by design documentation and initial testing.

- The highest-priority remaining work is data persistence (save/load) and improving test coverage for key services (alerts and reports).
- A small but important UI improvement is planned to improve usability: restricting categories based on transaction type (income vs expense), which will reduce user errors and improve the overall experience.

# 7 GUI Implementation (JavaFX) + Screenshots

This section presents the initial JavaFX GUI implementation of SmartBudget. The goal of the GUI in this iteration is to demonstrate the main user workflows through a clear interface and consistent navigation. The GUI is organized into three primary areas: **Transactions**, **Budgets**, and **Reports**. Each screen is connected to the service layer through controllers so that user actions (such as adding a transaction or setting a budget) immediately update what is shown on the screen. The screenshots in the subsections below provide evidence of the implemented pages and dialogs, and they show how the user can move through the system to complete the main tasks.

## 7.1 UI Structure and Navigation

The SmartBudget interface follows a simple multi-page layout. Users can navigate between the main pages (Transactions, Budgets, Reports) using the application's navigation controls. Each page focuses on one primary task and uses modal dialogs for create/edit actions to keep the workflow clear and avoid clutter. After the user submits a form (for example, adding a transaction or saving a budget), the system validates the inputs, performs the requested action through the service layer, and refreshes the page with updated data.

## 7.2 Transactions UI

### 7.2.1 Transactions Page

The Transactions page is the main workspace for viewing recorded transactions. It displays a list/table of transactions and provides controls for common actions such as adding a new transaction and selecting an existing transaction for editing or deletion. The page is designed to make it easy for users to review their recent activity and manage entries without leaving the Transactions module.

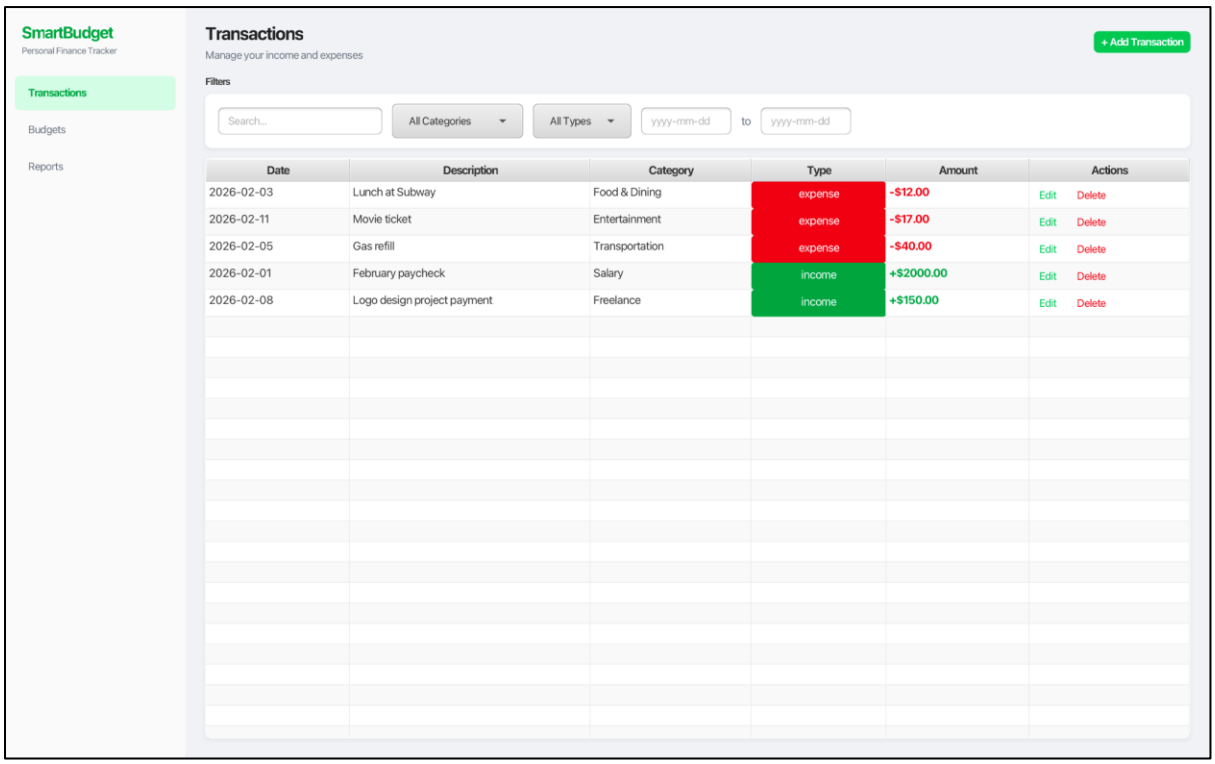


*Figure 7-1: Transactions Page (List/Table View)*

### 7.2.2 Add Transaction Modal

The Add Transaction modal allows the user to record a new transaction by entering the required details (for example: type, category, amount, date, and an optional note). When the user clicks "Save/Add," the controller sends the input to the service layer for validation and transaction creation. If the input is valid, the transaction is added and the Transactions page refreshes to show the new entry. If the input is invalid, the modal displays an error message and the transaction is not saved.

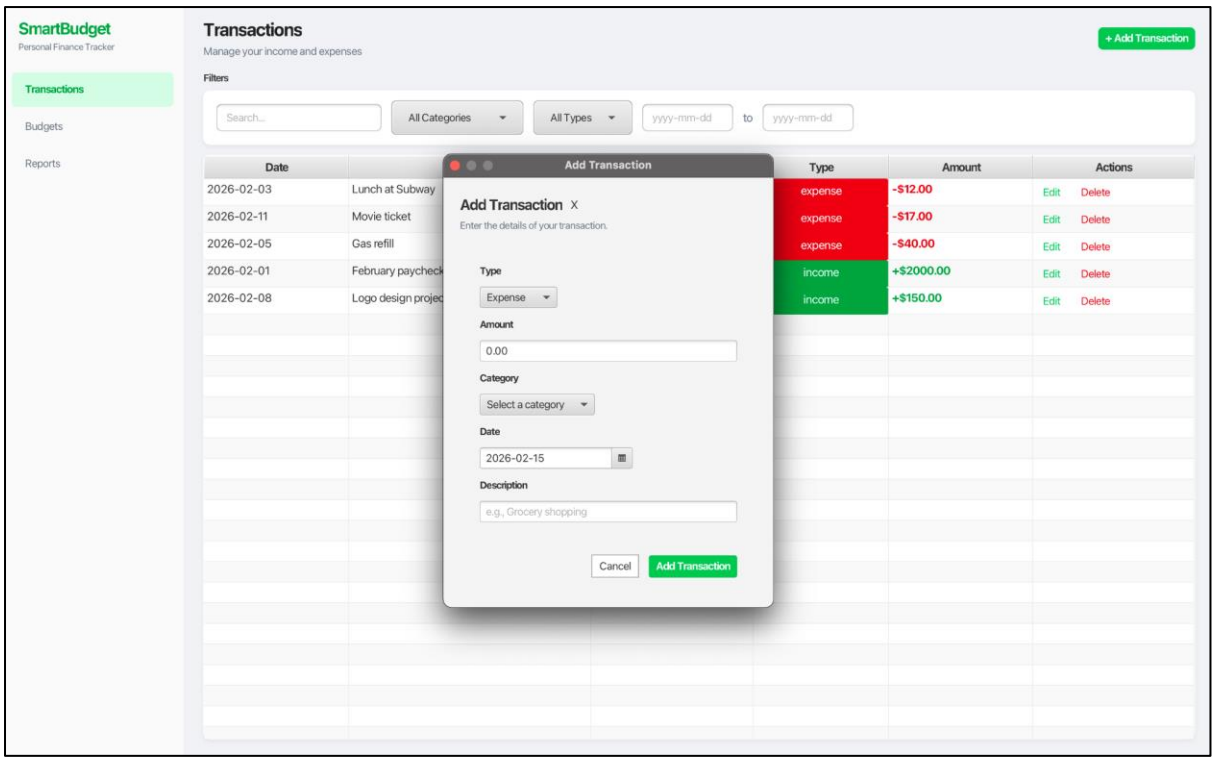


*Figure 7-2: Figure 8.2: Add Transaction Modal*

### 7.2.3 Edit Transaction Modal

The Edit Transaction modal is used to update an existing transaction. The user selects a transaction from the list and chooses the edit option. The modal opens with the current values pre-filled to reduce user effort and prevent mistakes. Once the user submits changes, the controller validates the updated fields through the service layer and then updates the transaction in the system. The Transactions page refreshes to reflect the updated information.

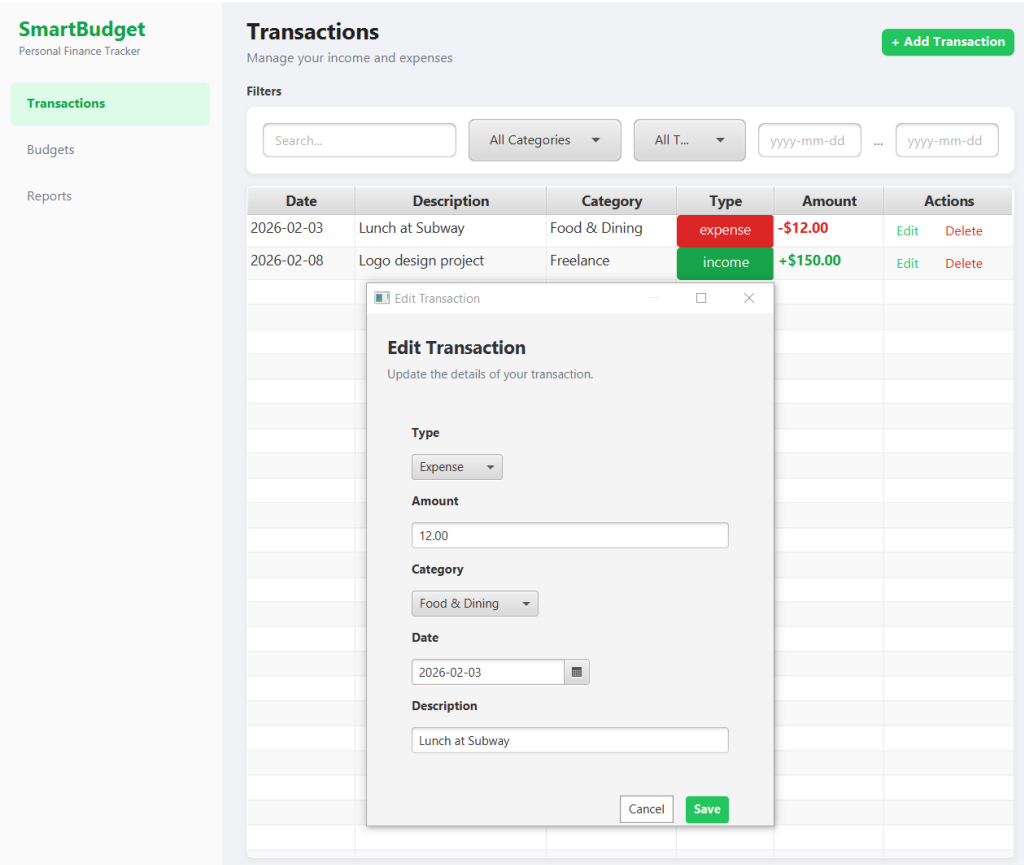


*Figure 7-3: Edit Transaction Modal (Pre-filled Fields)*

### 7.2.4 Delete Transaction Confirmation

The Delete Transaction flow is implemented as a confirmation dialog to prevent accidental removals. The user selects a transaction and chooses delete. The system prompts the user to confirm the action. If confirmed, the controller calls the service layer to remove the selected transaction, and the Transactions page refreshes so the deleted entry is no longer shown.

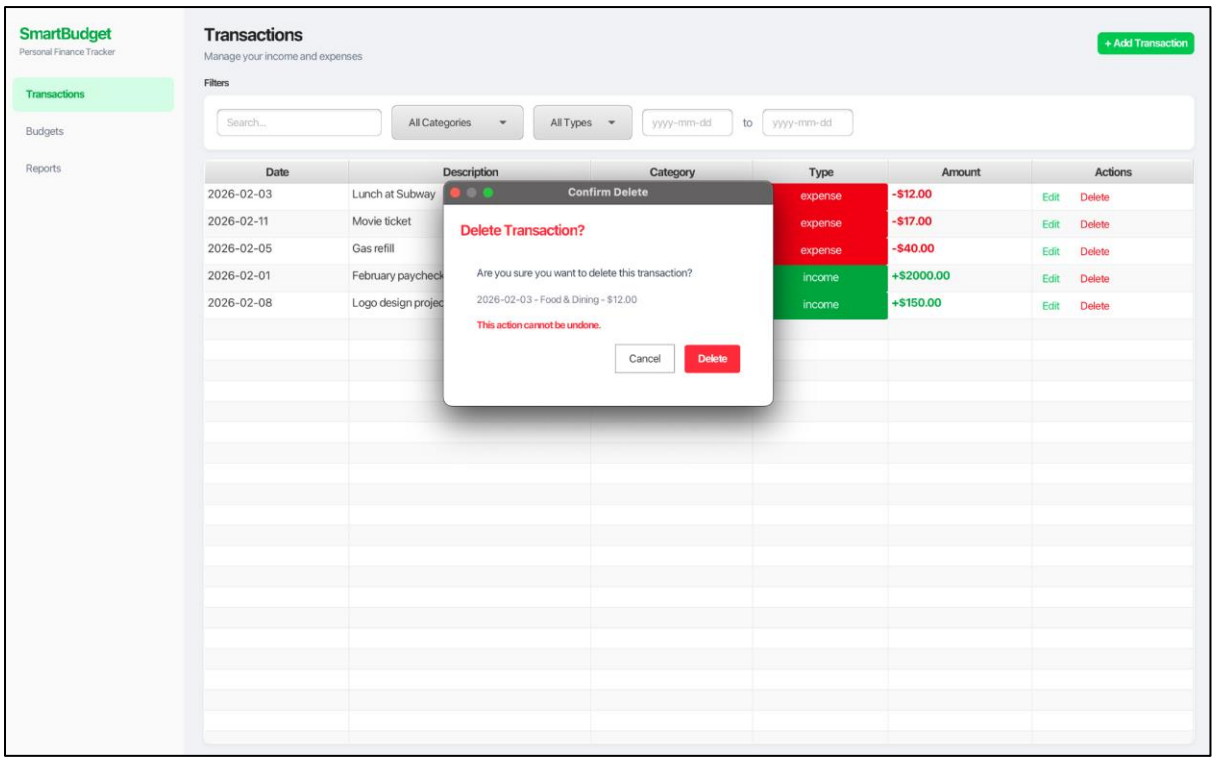


*Figure 7-4: Delete Transaction Confirmation Dialog*

## 7.3 Budgets UI

### 7.3.1 Budgets Page

The Budgets page allows the user to view and manage monthly budgets by category. The page shows existing budgets (such as category limits) and provides actions to add a new budget or update/delete an existing one. This page supports the budgeting workflow and helps the user compare planned spending (budgets) with actual spending (transactions).

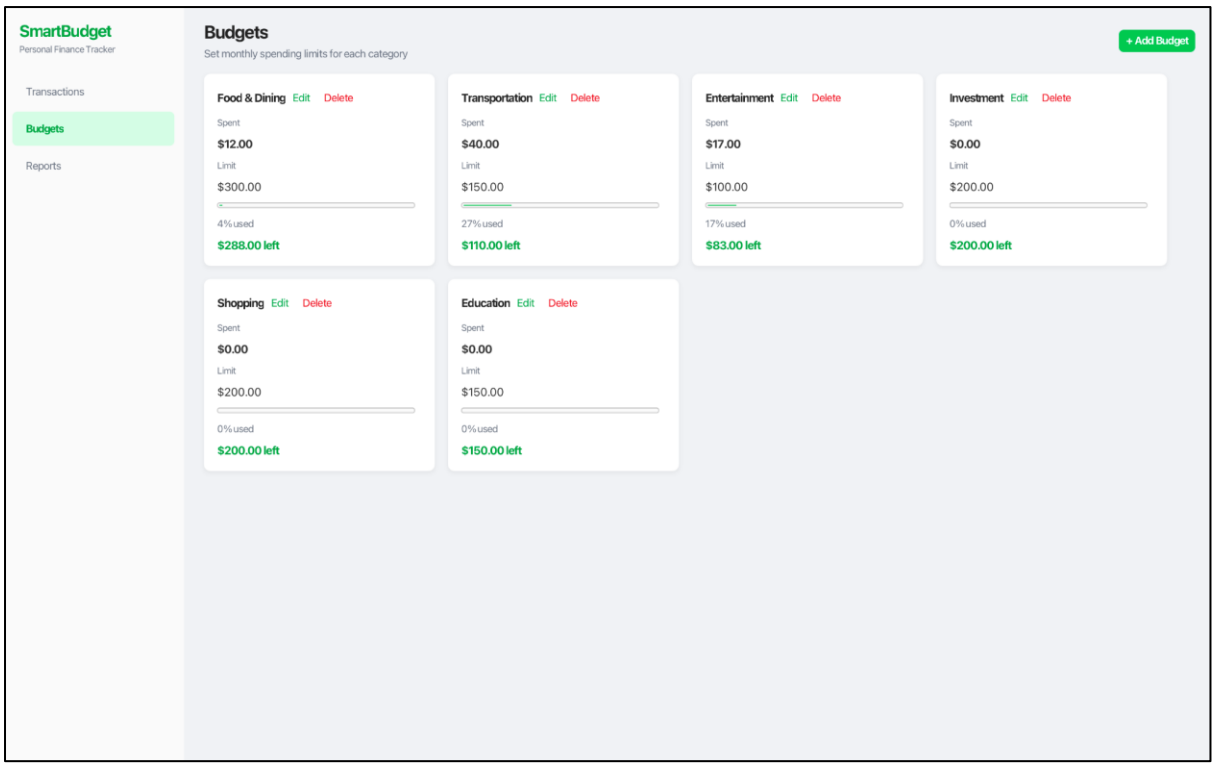


*Figure 7-5: Budgets Page*

### 7.3.2 Add Budget Modal

The Add Budget modal allows the user to create a monthly budget by selecting a category and entering a spending limit for a given month. When submitted, the controller validates the inputs (for example, checking valid numbers and required fields) and then saves the budget through the service layer. If the operation succeeds, the Budgets page refreshes to show the newly created budget.

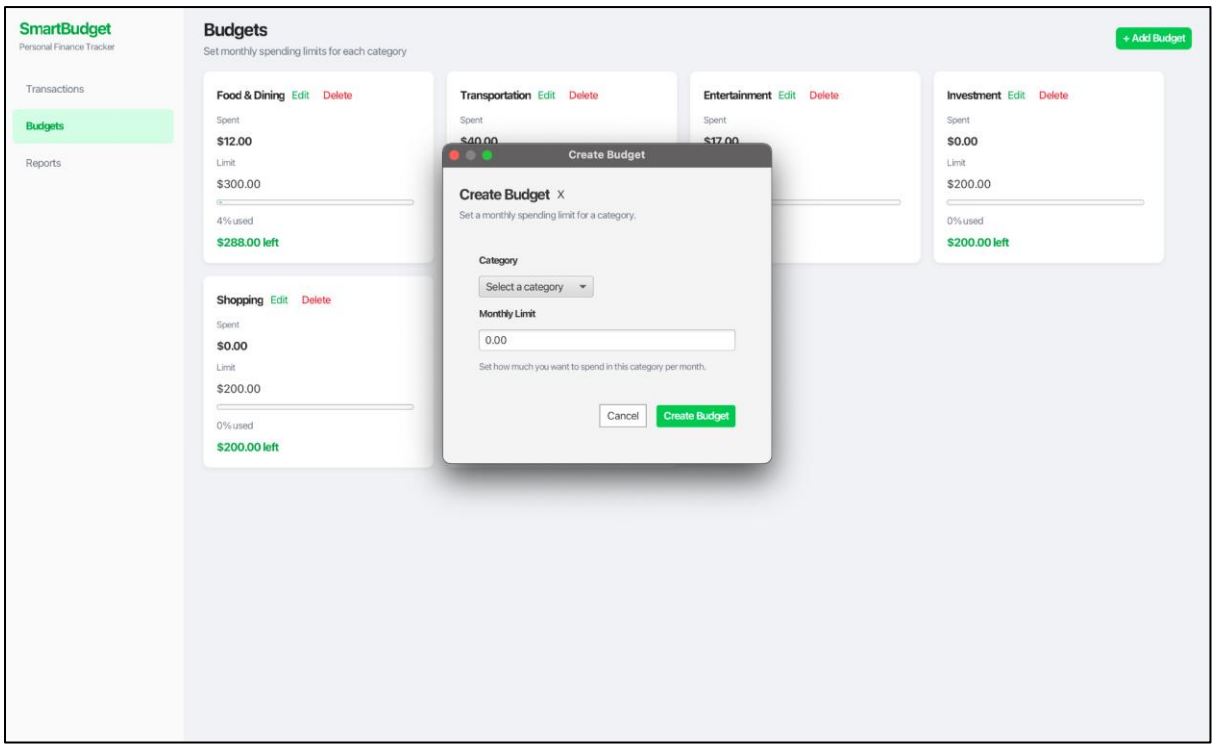


*Figure 7-6: Add Budget Modal*

### 7.3.3 Edit Budget Modal

The Edit Budget modal supports updating an existing budget limit. Similar to editing a transaction, fields are pre-filled with the current budget information. After submission, the service layer validates and updates the stored budget information, and the Budgets page refreshes to show the updated limit.

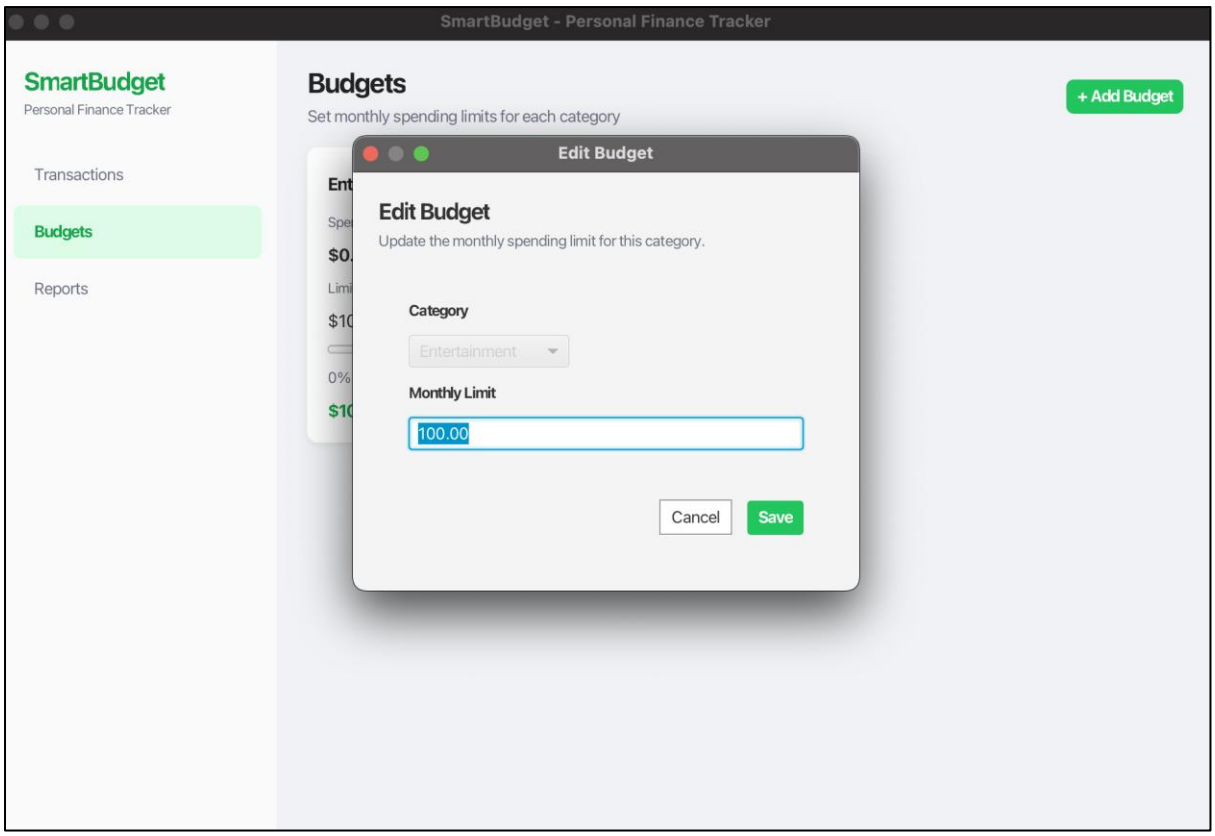


Figure 7-7: Edit Budget Modal

### 7.3.4 Delete Budget Confirmation

The Delete Budget flow is implemented with a confirmation dialog to avoid accidental deletions. After the user confirms deletion, the controller removes the selected budget through the service layer, and the Budgets page refreshes to reflect the change.

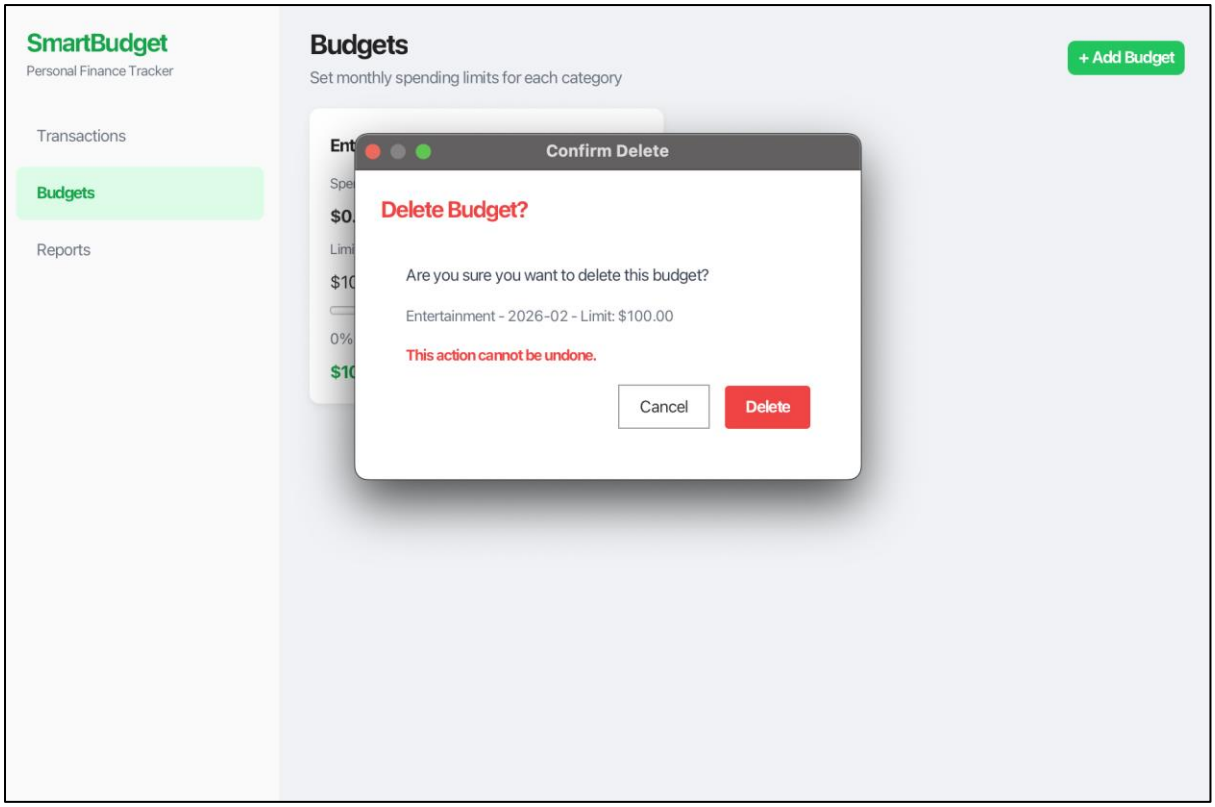


*Figure 7-8: Delete Budget Confirmation Dialog*

### 7.3.5 Reports UI

### 7.3.6 Summary Report Page

The Summary Report page allows users to generate and view spending summaries for a selected time period. The user chooses a date range (or report period), and the system generates a report that summarizes totals and category breakdowns using the report service. The output is displayed in a readable format so the user can quickly understand spending patterns. This page demonstrates the reporting workflow and how the UI consumes structured report results produced by the service layer.

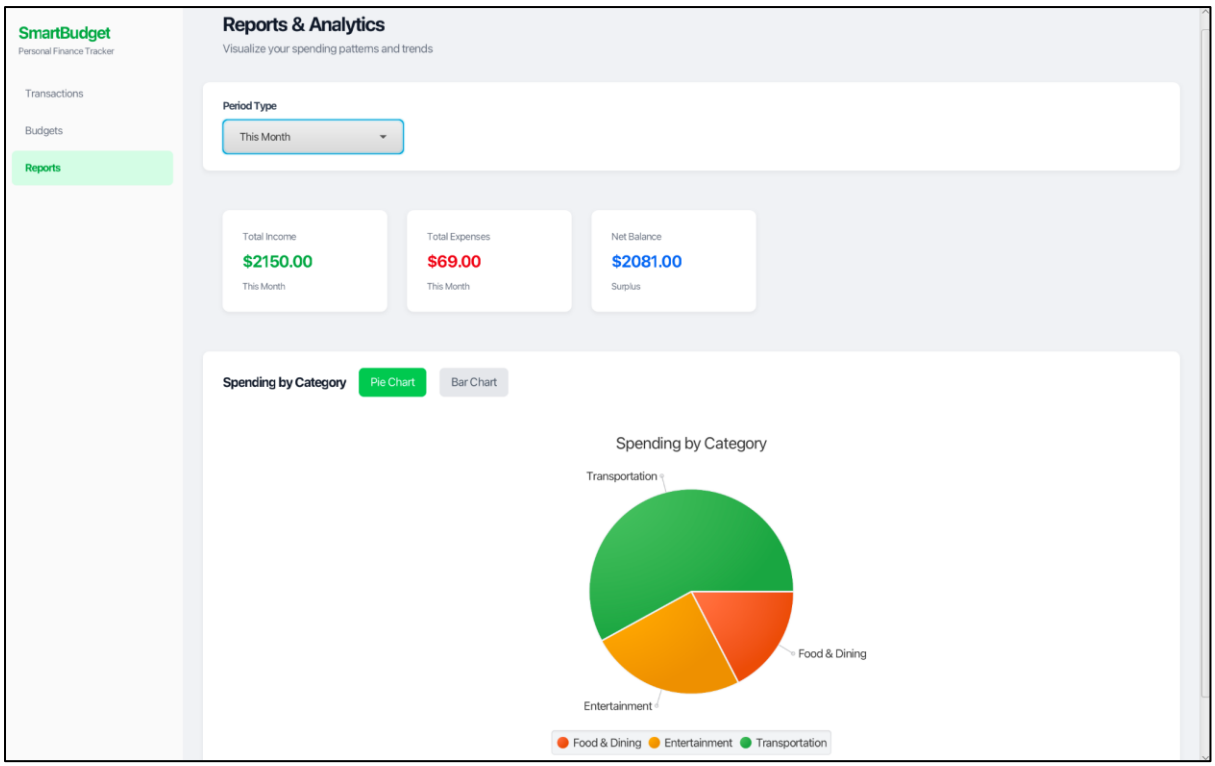


*Figure 7-9: Summary Report Page 1*

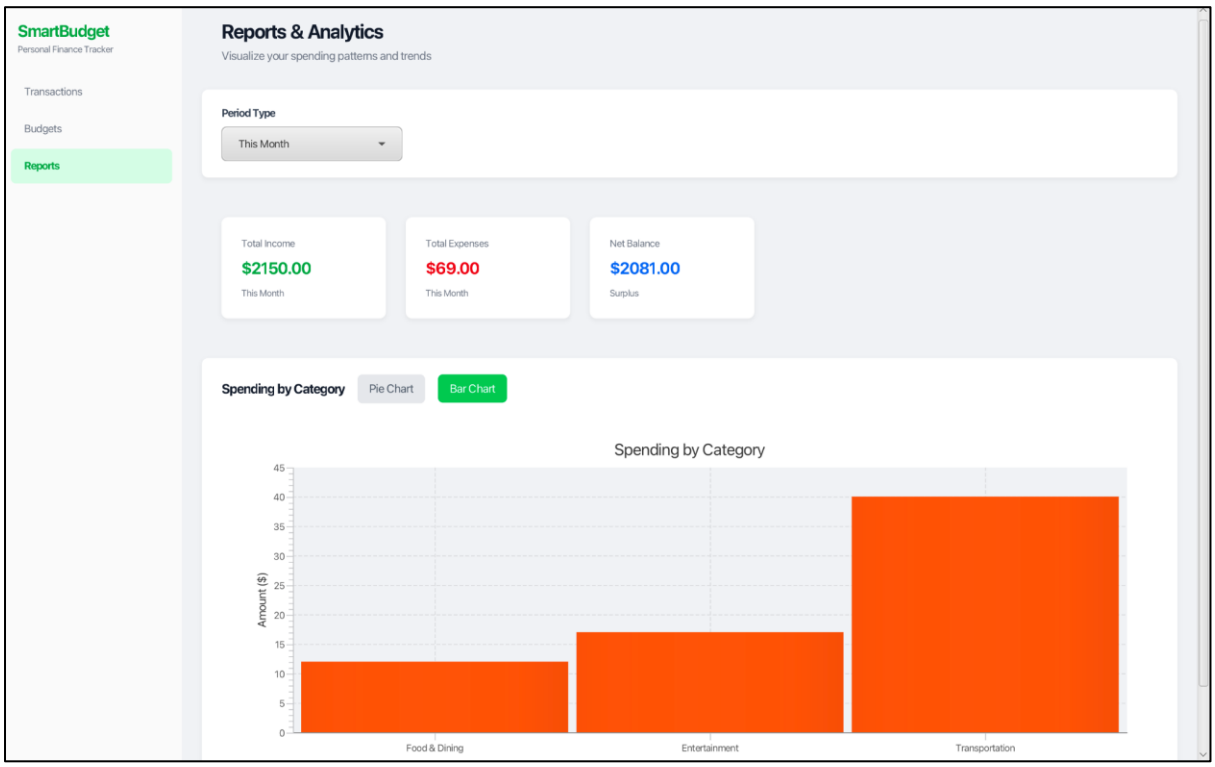


*Figure 7-10: Summary Report Page 2*

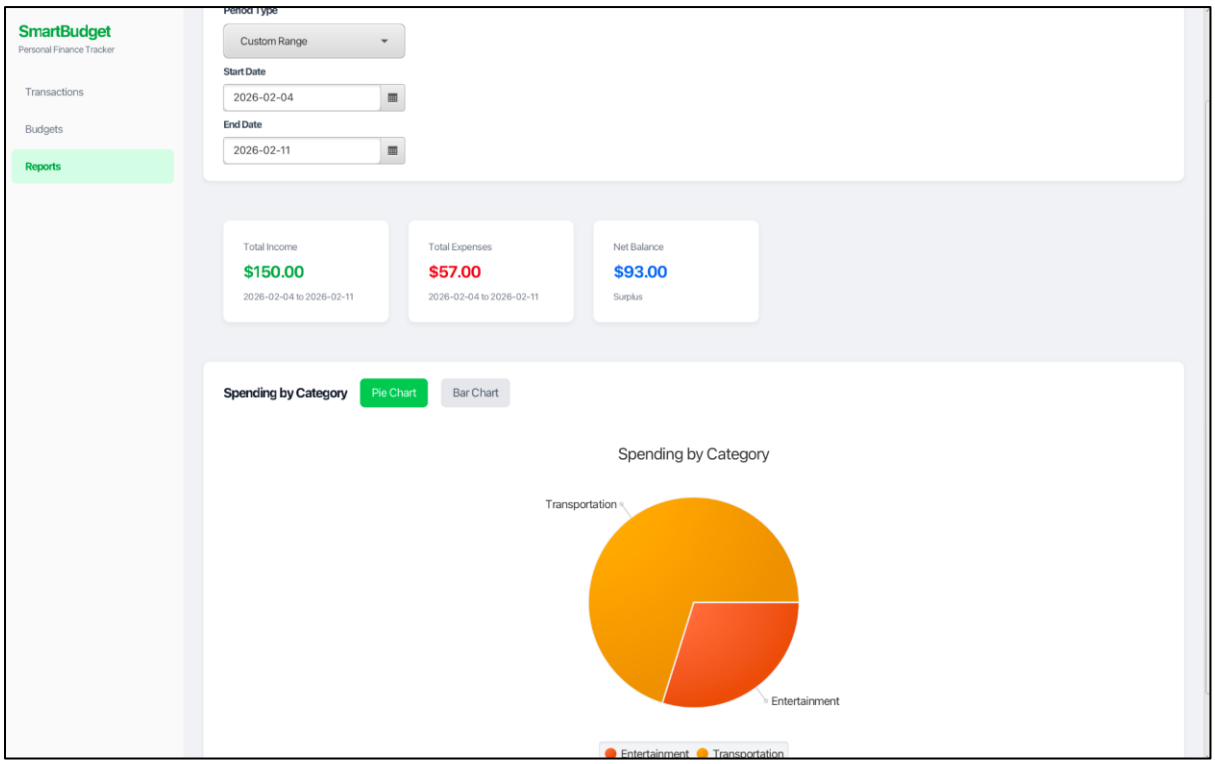


*Figure 7-11 3: Summary Report Page*

## 7.4 Notes on Current GUI Scope (Iteration 3.1)

In this iteration, the GUI is implemented to demonstrate the full workflow of the main features during an active session, including transactions, budgets, and report generation. The interface is connected to the business logic through controllers and services, and it supports validation and user feedback through dialogs. Further improvements (such as persistence integration and additional UI polish) will be completed in the next deliverable.

# 8 Preliminary Testing + JaCoCo Coverage

This section summarizes the initial testing completed in Deliverable 3.1. Since this iteration focuses on getting the main workflows working end-to-end, our testing includes:

- **Manual GUI smoke testing** to confirm the JavaFX screens behave correctly for the main user tasks, and
- **Automated unit testing (JUnit)** for the domain and service layers, supported by a JaCoCo coverage report.

The goal of this testing is to confirm that core features work as expected in a running session, and to identify coverage gaps that will be addressed in later development cycles (especially persistence and deeper service/report/alert testing).

## 8.1 Testing Approach

**Manual GUI** smoke testing was used to validate that the implemented screens and dialogs behave correctly (navigation, form submission, validation feedback, and basic CRUD operations). This is appropriate at this stage because desktop UI components are often validated through manual testing early on, while automated UI testing can be added later if needed. Evidence of the working GUI is also shown in Section 7 screenshots.

**Automated unit tests** were written for key domain and service classes to validate business logic and input validation rules without depending on the UI. JaCoCo was used to measure how much of the code base is executed by the unit tests and to highlight untested areas.

## 8.2 Manual GUI Smoke Test Results

The following smoke tests were executed to confirm the GUI operates as expected:

- Add transaction (valid input): Transaction is added and list refreshes — **Pass**
- Add transaction (invalid input): Error shown; transaction not saved — **Pass**
- Edit transaction: Updated values saved and reflected in list — **Pass**
- Delete transaction: Confirmation shown; transaction removed after confirm — **Pass**
- Filtering transactions: Filters update the displayed list correctly — **Pass**
- Add budget (valid input): Budget saved and page refreshes — **Pass**
- Add budget (invalid input): Error shown; budget not saved — **Pass**
- Edit budget / Delete budget: Changes reflected correctly after action — **Pass**
- Generate report for date range: Report output displayed for selected period — **Pass**
- Navigation between pages: User can move between Transactions, Budgets, and Reports pages — **Pass**

## 8.3 Automated Unit Testing Summary

- 55 unit tests were executed successfully (100% passing).

- The unit tests mainly target the validation logic, plus parts of the domain and service layers.
- This provides confidence that key business rules behave correctly independent of the GUI.

## 8.4 JaCoCo Coverage Summary

A JaCoCo coverage report was generated to measure test execution coverage across the code base. Overall coverage is currently low because the UI layer (controllers and modals) is not covered by unit tests in this iteration, which is common in early-stage desktop GUI projects.

- Overall coverage (all packages):
- Instruction coverage: 13%
- Branch coverage: 21%
- Line coverage: 17%
- Method coverage: 28%

*Table 8-1: JaCoCo Coverage Summary (by package)*

**SmartBudget**

| Element | Missed Instructions | Cov. | Missed Branches | Cov. | Missed | Cxty | Missed | Lines | Missed | Methods | Missed | Classes |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ca.yorku.smartbudget.ui | | 0% | | 0% | 147 | 147 | 562 | 562 | 63 | 63 | 8 | 8 |
| ca.yorku.smartbudget.ui.modals | | 0% | | 0% | 70 | 70 | 312 | 312 | 23 | 23 | 5 | 5 |
| ca.yorku.smartbudget.service | | 46% | | 37% | 70 | 106 | 87 | 190 | 11 | 29 | 2 | 4 |
| ca.yorku.smartbudget.domain | | 45% | | 26% | 60 | 94 | 73 | 135 | 30 | 56 | 5 | 11 |
| ca.yorku.smartbudget.util | | 70% | | 68% | 15 | 38 | 13 | 61 | 3 | 11 | 0 | 2 |
| ca.yorku.smartbudget | | 0% | | n/a | 3 | 3 | 16 | 16 | 3 | 3 | 1 | 1 |
| Total | 5,831 of 6,705 | 13% | 425 of 537 | 20% | 365 | 458 | 1,063 | 1,276 | 133 | 185 | 21 | 31 |

## 8.5 Issues Identified and How They Will Be Addressed

### 8.5.1 Main coverage gaps identified:

- UI layer is untested (0%): Controllers and JavaFX modals are not currently covered by automated tests. This is expected at this stage and is partially mitigated by the manual GUI smoke tests (Section 9.2) and GUI screenshots (Section 8).
- Alert-related logic is untested: *AlertService* (and related alert domain classes) currently show 0% coverage in the JaCoCo report. This means the overspending alert logic (UC-6) needs dedicated unit tests.

- Report generation is untested: *ReportService* and report result classes currently show 0% coverage. This indicates report logic (UC-5) should be validated with unit tests (including date range and empty-result cases).
- Persistence is not implemented yet: Since saving/loading is planned for the next deliverable, persistence behavior and startup loading tests are not included in this iteration.

### 8.5.2 Planned actions for the next development cycle:

- Add unit tests for AlertService to verify overspending detection and alert conditions (UC-6).
- Add unit tests for *ReportService* to validate summary calculations, category totals, date range handling, and empty results (UC-5).
- Expand domain-level tests for edge cases (invalid values, boundary dates, category totals, etc.).
- Implement persistence and add tests for: saving/loading correctness, startup load behavior, and regression tests to ensure existing workflows continue to work after persistence is added

# 9 Code Repository

The SmartBudget project is hosted on GitHub, and all project deliverables, documents, and source files are stored and updated there throughout each iteration.

## 9.1 Repository Link:

https://github.com/Hercules-Hawk/smart-budget-personal-finance-tracker

# 10 Conclusion

In this deliverable, our team completed the main design and implementation work for SmartBudget and brought the system to a solid, functional state. We produced the required software design documentation, including the layered architecture overview, UML class diagrams, and sequence/activity diagrams that clearly describe how the system is structured and how the key workflows operate. Based on these designs, we implemented the core features of the application, including transaction management, monthly budgeting, overspending alert

integration, and summary report generation, along with an initial JavaFX GUI that supports navigation and the main user interactions through pages and modal dialogs.

We also conducted preliminary testing to confirm that the GUI workflows behave as expected through manual smoke testing, and we developed automated JUnit unit tests for core business logic. A JaCoCo report was generated to measure current test coverage and highlight gaps, particularly in the UI layer and in service areas such as alerts and reporting that require more test cases.

Overall, SmartBudget meets the functional goals of this iteration and demonstrates the complete workflow of the system during an active session. The primary next step is implementing data persistence (save/load across sessions), along with expanding unit test coverage (especially for AlertService, ReportService, and date range logic) and making small UI improvements such as filtering categories based on transaction type. These improvements will be addressed in the next deliverable to further strengthen reliability, usability, and overall completeness of the system.